ФП_Матюшко_Вікторія

Nov 5, 2026 · @Sanya

Фінальний проєкт курсу Foundations of Cloud Computing: вебзастосунок **car-calculator** розгорнуто в AWS на **Amazon ECS (Fargate)** за **Application Load Balancer**, з базою **Amazon RDS for PostgreSQL**, автомасштабуванням, моніторингом CloudWatch і контролем витрат.

Застосунок	http://car-calc-alb-matiushko-368882875.us-east-1.elb.amazonaws.com
Health check API	/api/health → `{"status":"ok"}`
Код	github.com/Vi-encore/car-calculator, гілка `deploy/aws`
IaC-артефакти	`infra/aws/`: чеклист, task definitions, скрипт збірки
Регіон / акаунт	`us-east-1`, корпоративний акаунт GoIT (IAM user + MFA)

Про застосунок. Car-calculator оцінює ринкову ціну авто за маркою, моделлю, роками та регіоном: користувач реєструється, робить розрахунки, бачить історію та керує профілем з аватаркою. Стек: monorepo (Turborepo) з React 19 + Vite (frontend), NestJS 11 + Prisma 7 (backend), PostgreSQL. До цього проєкт працював на Vercel + Render + Neon; для фінального проєкту він **повністю перенесений в AWS** (фронт, бек, база і файли) у окремій гілці, без впливу на прод.

1. Опис середовища

Середовище — **Amazon ECS на AWS Fargate**: два сервіси (frontend і backend) за одним Application Load Balancer у власній VPC з двома зонами доступності; дані — у RDS PostgreSQL (приватні підмережі) і S3. Усі ресурси мають суфікс `-matiushko`, бо акаунт спільний для групи.

Шар	Ресурс	Ключові параметри
Мережа	VPC `car-calc-matiushko-vpc`	10.0.0.0/16, 2 AZ, 2 публічні + 2 приватні підмережі, Internet Gateway, без NAT

Шар	Ресурс	Ключові параметри
Мережа	Security groups	`car-calc-alb-sg-matiushko`, `car-calc-ecs-sg-matiushko`, `car-calc-rds-sg-matiushko`
Вхід	ALB `car-calc-alb-matiushko`	Internet-facing, HTTP:80; `/api/*` → backend TG, решта → frontend TG
Обчислення	ECS cluster `car-calc-cluster-matiushko`	Fargate, Container Insights
Обчислення	Service `car-calc-frontend-service-matiushko`	nginx, 0.25 vCPU / 0.5 GB, 1 таск
Обчислення	Service `car-calc-backend-service-matiushko`	NestJS, 0.5 vCPU / 1 GB, автомасштабування 1–3 таски
Образи	ECR `car-calc-backend-matiushko`, `car-calc-frontend-matiushko`	теги `latest` + git-SHA, scan on push
Дані	RDS `car-calc-db-matiushko`	PostgreSQL 17.11, db.t4g.micro, 20 GiB gp3, Single-AZ, Public access: No, шифрування
Дані	S3 `focc-car-calc-avatars-matiushko`	приватний бакет аватарок, Block Public Access
Секрети	Secrets Manager `focc-car-calc-backend-matiushko`	DATABASE_URL, DIRECT_URL, JWT-ключі, ключ Brevo
Доступ	IAM roles	`focc-ecs-task-execution-matiushko`, `focc-ecs-task-matiushko`, `focc-ec2-ecr-push-matiushko`
Спостереження	CloudWatch	2 log groups (retention 1 week), 3 алярми + 4 алярми автоскейлінгу, дашборд `car-calc-matiushko`
Сповіщення	SNS `car-calc-alerts-matiushko`	email-підписка (Confirmed)
Витрати	AWS Budgets `car-calc-budget-matiushko`	$10/міс, алерти 85 % / 100 % / прогноз
Збірка	EC2 `car-calc-builder-matiushko`	t3.medium, тимчасовий збирач образів (зупинений після збірки)

2. Обґрунтування вибору технологій

ECS Fargate обрано тому, що застосунок уже складається з двох контейнерів, а Fargate дає автомасштабування й інтеграцію з ALB без керування серверами.

Варіант	Рішення	Причина
ECS на Fargate	✅ обрано	Dockerfile під monorepo вже є; плата лише за vCPU/пам'ять тасків; нативна інтеграція з ALB, ECR, CloudWatch, Application Auto Scaling і IAM-ролями тасків; всі шари (VPC, SG, IAM) налаштовуються явно
Elastic Beanstalk	ні	ховає VPC, ALB і Auto Scaling за абстракцією; два сервіси в одному середовищі незручні
EKS	ні	control plane ≈ $73/міс лише за кластер; Kubernetes надлишковий для двох сервісів
EC2	ні	ручне керування ОС, Docker, оновленнями та Auto Scaling Group без виграшу для задачі

Роль кожного сервісу AWS:

Сервіс	Навіщо
VPC + підмережі	ізольована мережа; база фізично в приватних підмережах
Security Groups	фаєрвол ланцюжком інтернет → ALB → ECS → RDS
IAM	ролі для ECS і збирача — жодних ключів доступу у коді
ECR	приватний реєстр Docker-образів із скануванням вразливостей
ECS Fargate	запуск і перезапуск контейнерів, rolling-деплої з circuit breaker
ALB	єдина точка входу, маршрутизація за шляхом, health checks

Сервіс	Навіщо
RDS PostgreSQL	керована реляційна БД з бекапами і шифруванням (застосунок на Prisma + PostgreSQL)
S3	зберігання аватарок користувачів
Secrets Manager	пароль БД, JWT-ключі, API-ключі поза кодом і образом
CloudWatch + SNS	логи, метрики, дашборд, алярми з email-сповіщеннями
Application Auto Scaling	кількість backend-тасків за CPU і запитами
Budgets + Cost Explorer	контроль і аналіз витрат

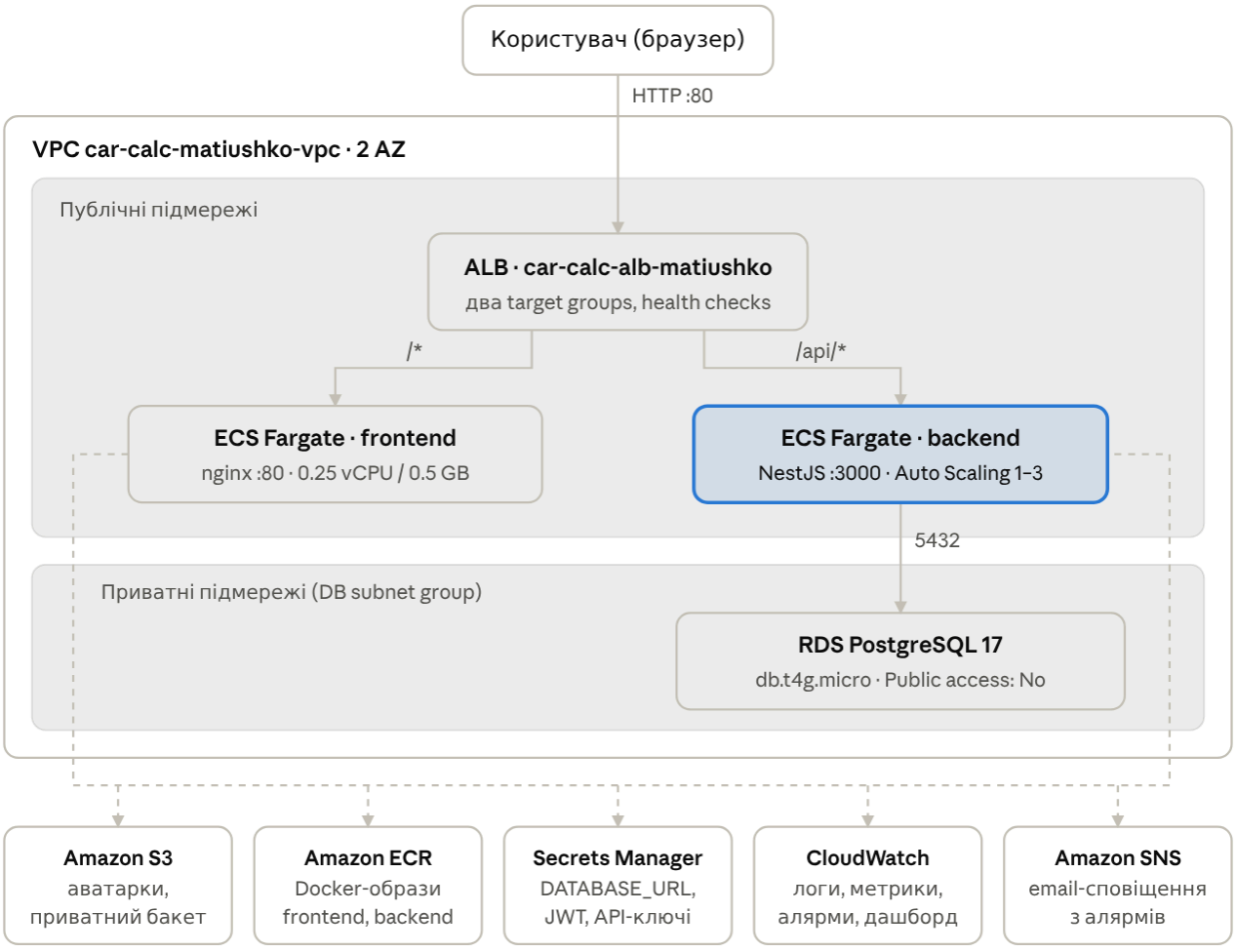
Ключові архітектурні рішення:

- **Один ALB для фронту й API** (правило `/api/*`): браузер бачить один origin, тож не потрібні CORS і крос-сайтові cookies; один балансувальник дешевший за два.
- **Два окремі образи** з одного monorepo (`turbo prune`): фронт — nginx зі статикою (21 MB), бек — NestJS (251 MB); масштабуються і деплояться незалежно.
- **Конфігурація через env** (12-factor): один код працює і на попередньому хостингу, і в AWS (`API_PREFIX`, `COOKIE_MODE`, `TRUST_PROXY`, `AVATAR_STORAGE=s3`, `GOOGLE_AUTH_ENABLED`).
- **Аватарки перенесено з Cloudinary у S3**, щоб усі дані застосунку були в AWS; доступ бекенду — через IAM task role.
- **Міграції Prisma при старті контейнера**: схема БД завжди відповідає версії коду; Prisma бере lock, тож кілька тасків безпечні.

Зовнішні залежності — лише сторонні API: Brevo (листи скидання пароля; SES недоступний у навчальному акаунті) і AUTO.RIA (ціни; у демо — mock-провайдер).

3. Архітектурна схема

Трафік іде лише ланцюжком інтернет → ALB → ECS → RDS



Регіональні сервіси AWS поза VPC — таски звертаються до них через IAM-ролі (без ключів)

архітектура · VPC, ALB, ECS Fargate, RDS і регіональні сервіси, us-east-1

Користувач бачить лише ALB: `/api/*` йде на backend, решта — на frontend. Backend звертається до RDS у приватних підмережах (5432, лише з SG ECS). Регіональні сервіси таски викликають через публічний IP (лише вихідний трафік, без NAT Gateway) з правами IAM-ролей: ECR (pull образів), Secrets Manager (секрети при старті), S3 (аватарки), CloudWatch (логи та метрики); алярми CloudWatch надсилають email через SNS. Application Auto Scaling змінює кількість backend-тасків (1–3) за метриками CloudWatch.

4. Безпека: VPC, Security Groups, IAM

Доступ побудований за принципом найменших привілеїв на трьох рівнях: маршрутизація (VPC), фаєрвол (SG), права сервісів (IAM).

VPC `car-calc-matiushko-vpc` (10.0.0.0/16, 2 AZ):

Підмережа	CIDR	Маршрут	Що там
public1 (us-east-1a)	10.0.0.0/20	→ Internet Gateway	ALB, ECS-таски
public2 (us-east-1b)	10.0.16.0/20	→ Internet Gateway	ALB, ECS-таски
private1 (us-east-1a)	10.0.128.0/20	лише local	RDS (DB subnet group)
private2 (us-east-1b)	10.0.144.0/20	лише local	RDS (DB subnet group)

Security Groups — ланцюжок довіри. Джерелом для ECS і RDS є інша security group, а не IP-діапазон, тож правила не залежать від IP тасків, які змінюються при кожному запуску.

Security group	Inbound	Джерело
`car-calc-alb-sg-matiushko`	HTTP 80	0.0.0.0/0
`car-calc-ecs-sg-matiushko`	TCP 3000, HTTP 80	лише SG ALB
`car-calc-rds-sg-matiushko`	PostgreSQL 5432	лише SG ECS
`car-calc-build-sg-matiushko`	SSH 22	лише prefix list EC2 Instance Connect

IAM-ролі (усі з префіксом `focc-` і permissions boundary `Foundations-of-Cloud-Computing-lab-scope`, як вимагає акаунт):

Роль	Trusted entity	Policies	Хто використовує
`focc-ecs-task-execution-matiushko`	ecs-tasks	`AmazonECSTaskExecutionRolePolicy`, `SecretsManagerReadWrite`	агент ECS: pull з ECR, логи, підстановка секретів
`focc-ecs-task-matiushko`	ecs-tasks	`AmazonS3FullAccess`	код бекенду: читання/запис аватарок
`focc-ec2-ecr-push-matiushko`	ec2	`AmazonEC2ContainerRegistryPowerUser`	EC2-збирач: push образів у ECR

Дані та секрети:

- Жодних access keys: сервіси отримують тимчасові креденшли ролей (task role, instance profile).
- Секрети (пароль БД, JWT, API-ключі) — у Secrets Manager (KMS); у task definition лише ARN, у коді та образах — нічого.
- RDS: Public access No, шифрування диску (KMS `aws/rds`), SSL-з'єднання.
- S3: Block Public Access, шифрування SSE-S3; файли віддає бекенд із перевіркою шляху; приймаються лише растрові формати (SVG — ризик XSS).
- ECR: scan on push. Застосунок: helmet, rate limiting з реальним IP клієнта (`X-Forwarded-For` від ALB), httpOnly refresh-cookie.

5. Інструкція з деплою та налаштувань

Усе створено через AWS Console в `us-east-1`. Детальний чеклист із усіма значеннями полів — infra/aws/README.md; нижче — порядок дій.

Підготовка коду (гілка `deploy/aws`):

1. Frontend-образ: `turbo prune` → Vite-збірка з `VITE_API_URL=/api` → nginx (SPA fallback, `/health`).
2. Backend: глобальний префікс `/api`, `GET /api/health`, довіра до ALB (`X-Forwarded-For`), same-site cookies для HTTP, S3-сховище аватарок, вимикач Google OAuth. Все — через env-змінні з валідацією (zod), дефолти зберігають стару поведінку; типи, lint і тести — зелені.

Інфраструктура (по порядку залежностей):

1. **S3** — бакет аватарок, Block all public access.
2. **IAM** — ролі `focc-ecs-task-execution-*` і `focc-ecs-task-*` (use case: Elastic Container Service Task, boundary курсу).
3. **VPC** — майстер *VPC and more*: 2 AZ, 2 public + 2 private, NAT None, VPC endpoints None.
4. **Security groups** — ALB → ECS → RDS (див. розділ 4).
5. **RDS** — DB subnet group з приватних підмереж, PostgreSQL 17, db.t4g.micro, 20 GiB gp3, Single-AZ, Public access No, initial DB `car_calculator`, Self managed password.
6. **Secrets Manager** — секрет з `DATABASE_URL` (`sslmode=no-verify`), `DIRECT_URL` (`sslmode=require`), JWT-ключами (`openssl rand -hex 32`), `BREVO_API_KEY`.
7. **CloudWatch Logs** — log groups `/ecs/car-calc-{backend,frontend}-matiushko`, retention 1 week.
8. **ECR** — два репозиторії, Mutable, scan on push.

9. **Збірка образів** — на тимчасовому EC2 (Amazon Linux 2023, t3.medium, роль з `AmazonEC2ContainerRegistryPowerUser`, Docker і git через user data), підключення EC2 Instance Connect:

   ```
   git clone -b deploy/aws https://github.com/Vi-encore/car-calculator.git
   && cd car-calculator
   nohup bash infra/aws/build-and-push.sh > ~/build.log 2>&1 &
   ```

   Скрипт логіниться в ECR (`aws ecr get-login-password`), збирає обидва образи та пушить теги `latest` і git-SHA.
10. **ALB** — два target groups (type IP; health check `/api/health` і `/health`), internet-facing ALB у публічних підмережах, listener HTTP:80 → frontend, правило `/api/*` (priority 10) → backend.
11. **ECS** — кластер (Fargate, Container Insights), task definitions з JSON (backend, frontend), два сервіси: Fargate, публічні підмережі, SG ECS, Public IP on, existing ALB + target group, health check grace 30 s / 120 s.
12. **Auto Scaling, SNS, алярми, дашборд, Budgets** — див. розділи 6–7.

Оновлення застосунку: на збирачі `git pull && bash infra/aws/build-and-push.sh` (або `APPS=frontend` для одного образу) → ECS → сервіс → **Update → Force new deployment**. Міграції БД застосовуються автоматично при старті нового таска.

Перевірка: `/api/health` → 200, таргети обох target groups — Healthy, у логах бекенду — «All migrations have been successfully applied» і «Nest application successfully started»; реєстрація, вхід, розрахунок, історія і завантаження аватарки в S3 працюють.

6. Моніторинг, автомасштабування й балансування

Під навантаженням ~1 300 req/s backend сам масштабувався з 1 до 3 тасків, а ланцюжок алярм → SNS → email перевірено на реальних метриках.

6.1. Балансування (ALB)

- Listener HTTP:80; правило priority 10 `Path /api/*` → `car-calc-backend-tg-matiushko` (IP, :3000), default → `car-calc-frontend-tg-matiushko` (IP, :80).
- Health checks: `/api/health` і `/health`, interval 15 s, healthy threshold 2. Нездоровий таск виводиться з ротації, ECS замінює його новим.
- ALB і таски в двох AZ; ECS реєструє нові таски в target group автоматично.

6.2. Автомасштабування (Application Auto Scaling)

Сервіс `car-calc-backend-service-matiushko`, діапазон **1–3 таски**, дві target-tracking політики (scale-out cooldown 60 s, scale-in 120 s):

Політика	Метрика	Ціль
`backend-cpu-50`	ECSServiceAverageCPUUtilization	50 %
`backend-requests-per-target`	ALBRequestCountPerTarget	1000 запитів/хв на таск

CPU ловить важкі запити (bcrypt, розрахунки), RequestCountPerTarget — обсяг трафіку навіть на легкі ендпоінти. AWS сам створив для них 4 алярми (AlarmHigh/AlarmLow).

6.3. Моніторинг (CloudWatch + SNS)

- **Логи:** awslogs-драйвер → `/ecs/car-calc-{backend,frontend}-matiushko`, retention 1 week (структуровані JSON-логи pino, токени та cookies замасковані).
- **Метрики:** ALB, ECS + Container Insights, RDS (Database Insights Standard).
- **Дашборд `car-calc-matiushko`:** ALB RequestCount + TargetResponseTime; ECS CPU/Memory обох сервісів; RunningTaskCount; RDS CPU + DatabaseConnections.
- **SNS:** топік `car-calc-alerts-matiushko`, email-підписка Confirmed.
- **Алярми зі сповіщенням (ALARM і OK → SNS):**

Алярм	Умова	Що ловить
`car-calc-backend-unhealthy-matiushko`	UnHealthyHostCount ⩾ 1, 2 з 2 хв	бекенд впав
`car-calc-alb-latency-matiushko`	TargetResponseTime > 2 s (avg, 5 хв)	деградація швидкодії
`car-calc-rds-cpu-high-matiushko`	RDS CPU > 80 % (avg, 5 хв)	перевантаження бази

6.4. Навантажувальне тестування

Інструмент — autocannon з AWS CloudShell, 50 одночасних з'єднань на `GET /api/health` через ALB (повний шлях до NestJS, без rate limiter-а і зовнішніх API).

Показник	Тест 1 (старт з 1 таска)	Тест 2 (3 таски)
Тривалість	600 s	420 s
Запитів	796 275	863 166
Середня пропускна здатність	1 327 req/s	2 055 req/s
Latency: медіана / p99 / max	27 / 193 / 2 657 ms	13 / 187 / 890 ms
Non-2xx	9 (0.001 %)	3

Тест 1 — автомасштабування. CPU єдиного таска зріс до ~100 %, обидва AlarmHigh перейшли в ALARM; за Events сервісу: 23:25 *Successfully set desired count to 3* (причина — AlarmHigh політики `backend-requests-per-target`) → *started 2 tasks* → 23:26 *registered targets* → 23:27 *steady state*. Після навантаження AlarmLow (15 хв нижче цілі) повернув сервіс до 1 таска у два кроки: 00:02 — 3 → 2, 00:05 — 2 → 1 (з draining з'єднань перед зупинкою тасків).

Тест 2 — сповіщення. У тесті 1 система впоралась, тож алярми зі сповіщенням залишились OK. Щоб перевірити ланцюжок сповіщень на реальних даних, поріг latency-алярму тимчасово знижено до 20 ms: фактичне середнє ALB (25.6 ms) перевищило поріг → 21:38 UTC ALARM → email «ALARM: car-calc-alb-latency-matiushko»; після повернення порога 2 s → 21:43 UTC OK → email «OK».

Висновок: горизонтальне масштабування збільшило пропускну здатність на ~55 % і вдвічі знизило медіанну затримку. Для продакшну варто тримати min 2 таски (по одному в кожній AZ), щоб уникнути піку затримки під час scale-out.

7. Оптимізація витрат і прогноз

Прогноз вартості деплою — ≈ $2.6 на день (≈ $78 на місяць) у базовому стані (1 backend-таск); після оцінювання ресурси видаляються. Розрахунок за on-demand цінами `us-east-1` (наближено, без податків і кредитів).

Стаття витрат	Розрахунок	≈ $/міс
ECS Fargate	backend 0.5 vCPU / 1 GB + frontend 0.25 vCPU / 0.5 GB, 24/7	27.0
Application Load Balancer	$0.0225/год + LCU (при низькому трафіку — центи)	16.5

Стаття витрат	Розрахунок	≈ $/міс
Public IPv4	4 адреси (2 — ALB, 2 — таски) × $0.005/год	14.6
RDS PostgreSQL	db.t4g.micro Single-AZ + 20 GiB gp3	14.0
CloudWatch, Secrets Manager, ECR, S3	7 алярмів, Container Insights, логи; 1 секрет; ~0.3 GB образів	3.5
EC2-збирач (зупинений)	лише диск 30 GiB gp3	2.4
Разом		**≈ 78**

Прогноз за сценаріями:

Сценарій	День	Тиждень	Місяць
Базовий (1 backend-таск, збирач зупинено)	$2.6	$18	$78
Постійне навантаження (3 backend-таски 24/7)	$3.9	$27	$117
Демо на час оцінювання (~7 днів) → видалення	—	**≈ $18 разово**	$0 після прибирання
Оптимізований продакшн (див. нижче)	≈ $1.8	≈ $13	≈ $55

Навантажувальні тести коштували майже нічого: 2 додаткові таски близько години ≈ $0.05 + LCU ALB за ~1.7 млн запитів — кілька центів.

Що зроблено для економії (порівняно з дефолтами AWS):

Рішення	Економія
Без NAT Gateway (таски в публічних підмережах, вхід лише від ALB)	≈ $33/міс + трафік
RDS 20 GiB замість дефолтних 200 GiB	≈ $21/міс
RDS замість Aurora (дефолт форми — db.r7g.large)	сотні $/міс
db.t4g.micro (Graviton), Single-AZ, без Enhanced Monitoring	Multi-AZ подвоїв би ціну

Рішення	Економія
Автомасштабування 1–3 замість постійних 3 тасків	≈ $36/міс
Логи з retention 1 week, дашборд у безкоштовному ліміті, Database Insights Standard	витрати на моніторинг не ростуть
EC2-збирач увімкнений лише на час збірки	≈ $30/міс (t3.medium 24/7)

Контроль витрат:

- **AWS Budgets:** власний бюджет `car-calc-budget-matiushko` — $10/міс, email-алерти при 85 % і 100 % фактичних витрат та 100 % прогнозу, отримувач Verified. Бюджет без фільтрів рахує весь спільний акаунт курсу ($70.56 за листопад, $351 за жовтень), тому одразу показує перевищення; у власному акаунті його звужують тегом `Project = car-calculator` (cost allocation tags).
- **AWS Cost Explorer:** витрати за сервісами (Daily, Group by Service). Підсумок ≈ $0.00, бо кредити курсу покривають витрати, а дані надходять із затримкою до 24 год; для аналізу реального споживання виключається Charge type = Credit.

Оптимізований продакшн (≈ $55/міс): статика фронтенду з S3 + CloudFront замість ECS-сервісу (−$9 Fargate, −$3.6 IPv4), Compute Savings Plan на 1 рік для Fargate (−20 %), Reserved Instance для RDS (−30 %), Fargate Spot для додаткових тасків автоскейлінгу, scheduled scaling на нічні години, CI/CD замість EC2-збирача.

8. Обмеження і прийняті рішення

Навчальний акаунт і бюджет зумовили кілька компромісів; для кожного — як було б у продакшні.

Обмеження	Рішення в проєкті	У продакшні
IAM-ролі лише `focc-…` з boundary, без власних policy	керовані AWS policy, права обрізає boundary	inline least-privilege: `s3:GetObject/PutObject` на `<bucket>/avatars/*`, `secretsmanager:GetSecretValue` на один секрет

Обмеження	Рішення в проєкті	У продакшні
Немає access keys	збірка на EC2 з instance profile; застосунок — task role	CI/CD: GitHub Actions + OIDC-роль, автодеплой
Docker у CloudShell працює зі storage driver `vfs` — збірка monorepo не вміщається (`no space left on device` при 9 GB вільних)	тимчасовий EC2-збирач (`overlay2`, 30 GiB)	CodeBuild або CI-ранер
NAT Gateway ≈ $33/міс	ECS-таски в публічних підмережах з публічним IP; вхідний трафік лише від ALB (SG); БД — у приватних	таски в приватних підмережах за NAT або VPC endpoints
Немає домену і сертифіката	HTTP за DNS-ім'ям ALB; refresh-cookie у режимі same-site без `Secure`	Route 53 + ACM, HTTPS-listener, редірект 80 → 443, `Secure`-cookies
Google OAuth вимагає HTTPS-домен для redirect URI	кнопка неактивна з поясненням, `/api/auth/google*` → 404; вхід email + пароль	увімкнути після підключення домену
CA RDS не довірений у Node	рантайм `sslmode=no-verify` (шифрування є, трафік у VPC), міграції `sslmode=require`	RDS CA bundle у образі + `sslmode=verify-full`
Enhanced Monitoring і RDS log exports потребують ролей без префікса `focc-`	вимкнено; базові метрики + Database Insights Standard	увімкнути Enhanced Monitoring і експорт логів PostgreSQL
Спільний акаунт для групи	суфікс `-matiushko` у назвах; бюджет і Cost Explorer показують витрати всього акаунта	окремий акаунт (Organizations) або cost allocation tags

Обмеження	Рішення в проєкті	У продакшні
Немає кропу зображень у S3 (був у Cloudinary)	зберігається оригінал до 5 MB	ресайз у бекенді або Lambda на подію S3, віддача через CloudFront
Безкоштовний ліміт AUTO.RIA вичерпаний	`PRICES_PROVIDER=mock`	реальний провайдер вмикається однією env-змінною

Прибирання після оцінювання (зворотний порядок): ECS-сервіси → кластер → ALB і target groups → RDS (без final snapshot) і subnet group → EC2-збирач → секрет → ECR → S3 (Empty → Delete) → log groups, алярми, дашборд, SNS → security groups → VPC.

9. Скріншоти

Повний набір (64 знімки) — у репозиторії: [docs/aws/screenshots](). Нижче — ключові.

9.1. Розгортання середовища

Мережа і безпека

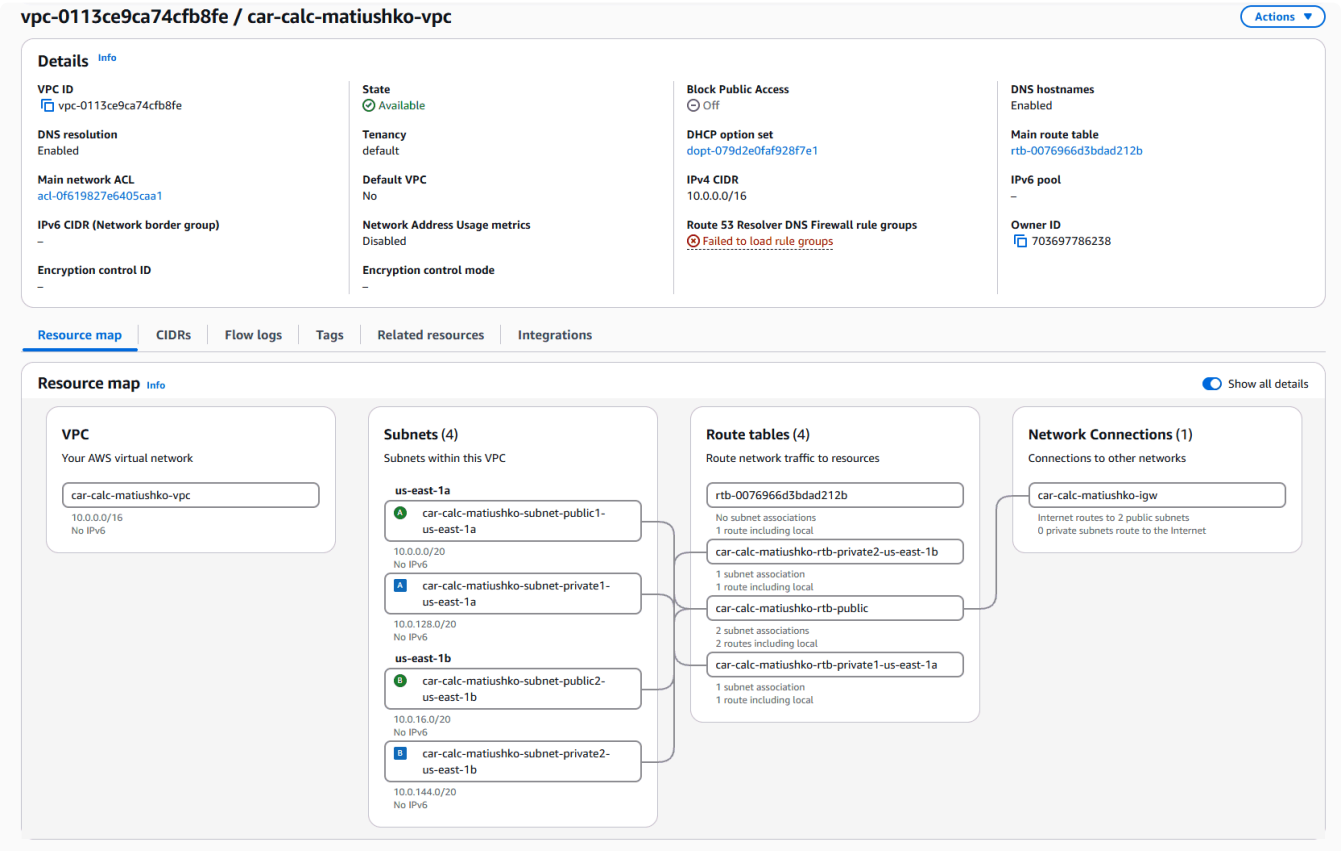


Рис. 1. VPC `car-calc-matiushko-vpc` (Resource map): чотири підмережі в двох AZ. Публічна route table веде на Internet Gateway, приватні мають лише маршрут local — база недосяжна з інтернету.

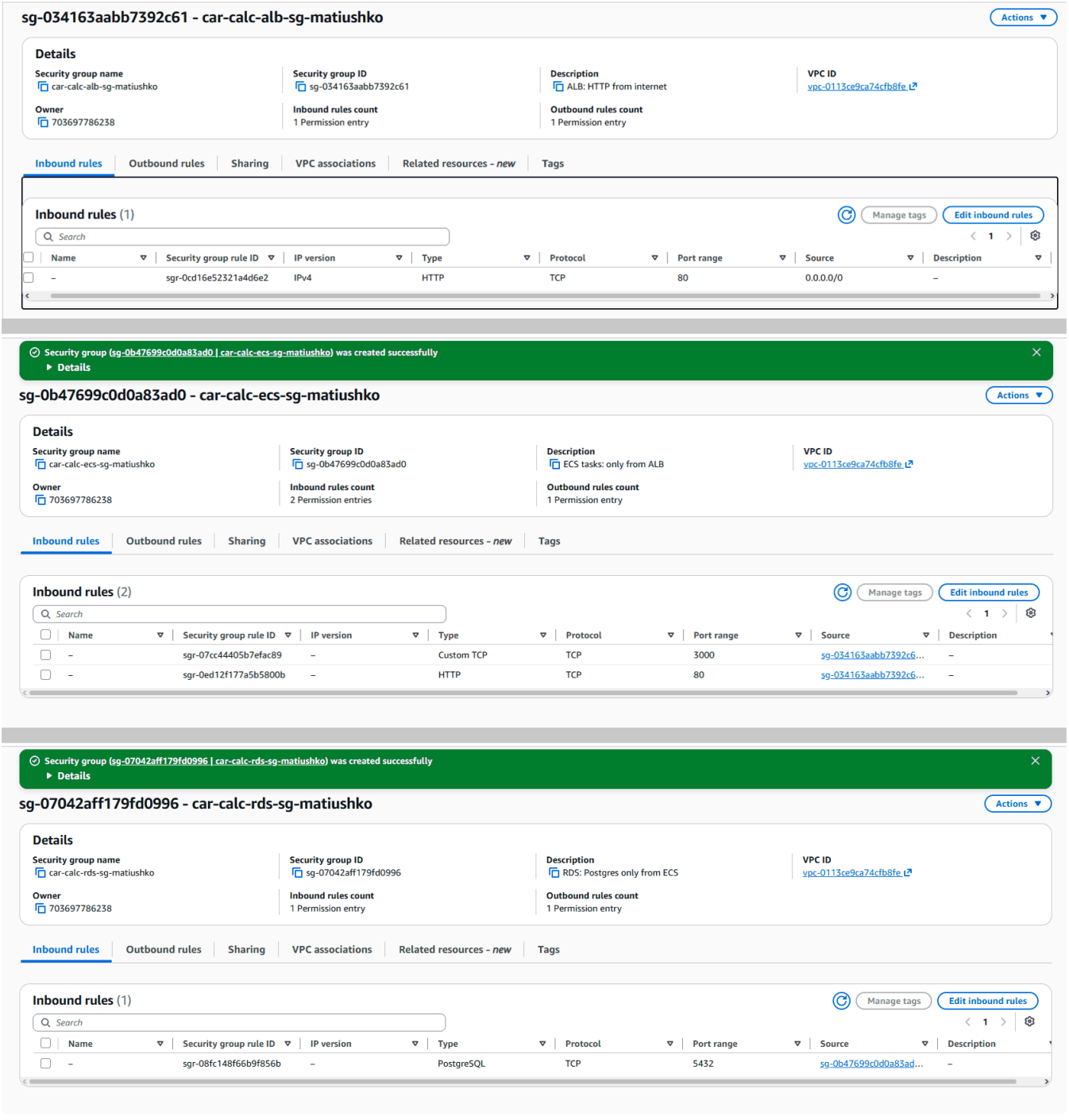
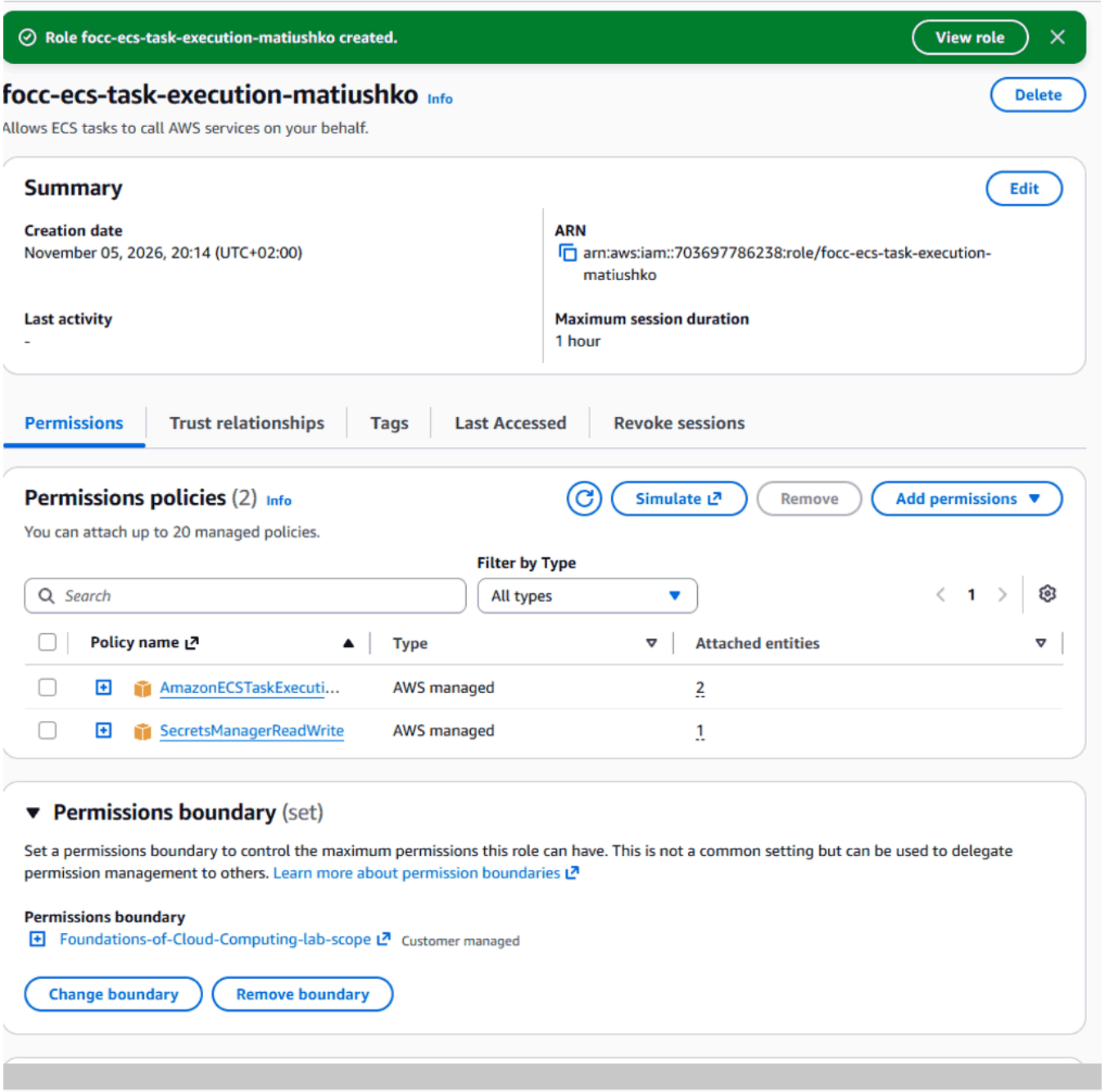
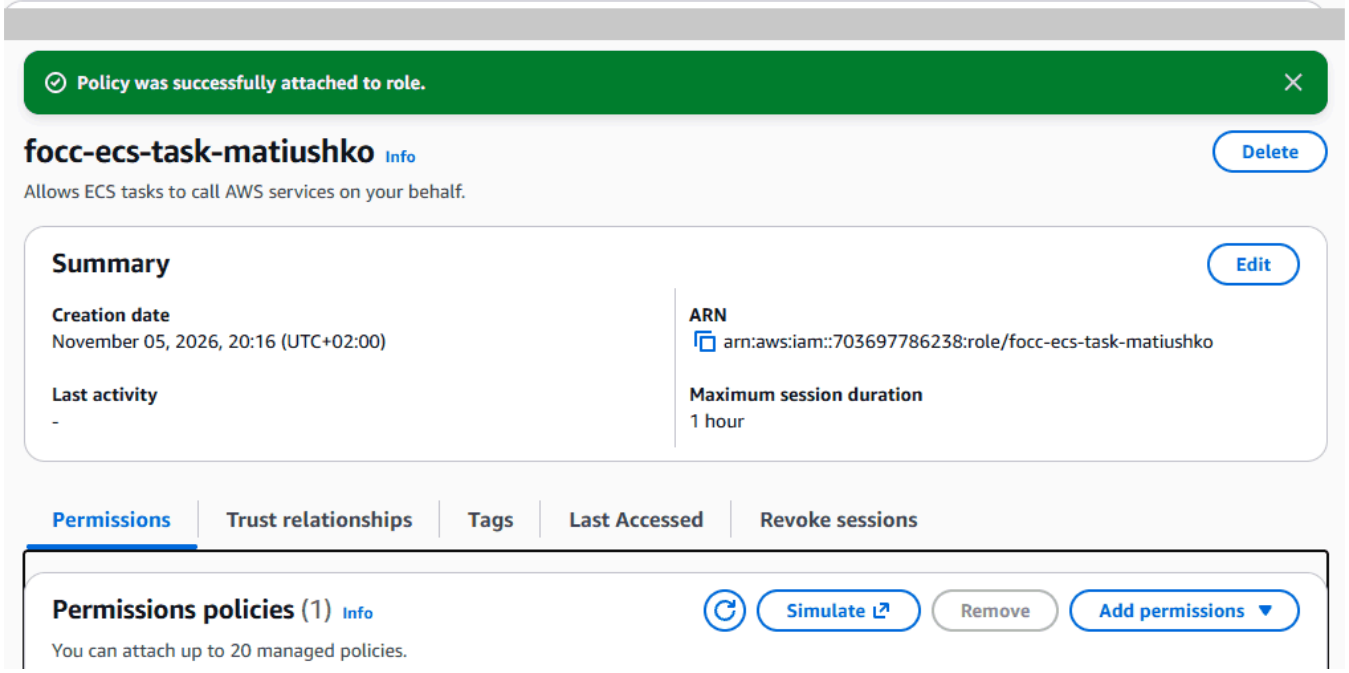


Рис. 2. Security group ALB: єдине правило — HTTP 80 з інтернету (0.0.0.0/0). Це єдина точка входу в систему.

Рис. 3. Security group ECS-тасків: порти 3000 (backend) і 80 (frontend) відкриті лише для SG балансувальника, тож напряму до тасків звернутися не можна.

Рис. 4. Security group RDS: PostgreSQL 5432 приймається лише від SG ECS-тасків — остання ланка ланцюжка інтернет → ALB → ECS → RDS.


Role focc-ecs-task-execution-matiushko created.
View role
focc-ecs-task-execution-matiushko Info
Delete
Allows ECS tasks to call AWS services on your behalf.
Summary
Edit
Creation date
November 05, 2026, 20:14 (UTC+02:00)
ARN
arn:aws:iam::703697786238:role/focc-ecs-task-execution-matiushko
Last activity
-
Maximum session duration
1 hour
Permissions
Trust relationships
Tags
Last Accessed
Revoke sessions
Permissions policies (2) Info
Simulate
Remove
Add permissions
You can attach up to 20 managed policies.
Filter by Type
Search
All types
1
Policy name
Type
Attached entities
AmazonECSTaskExecuti...
AWS managed
2
SecretsManagerReadWrite
AWS managed
1
Permissions boundary (set)
Set a permissions boundary to control the maximum permissions this role can have. This is not a common setting but can be used to delegate permission management to others. Learn more about permission boundaries
Permissions boundary
Foundations-of-Cloud-Computing-lab-scope
Customer managed
Change boundary
Remove boundary


Policy was successfully attached to role.
focc-ecs-task-matiushko Info
Delete
Allows ECS tasks to call AWS services on your behalf.
Summary
Edit
Creation date
November 05, 2026, 20:16 (UTC+02:00)
ARN
arn:aws:iam::703697786238:role/focc-ecs-task-matiushko
Last activity
-
Maximum session duration
1 hour
Permissions
Trust relationships
Tags
Last Accessed
Revoke sessions
Permissions policies (1) Info
Simulate
Remove
Add permissions
You can attach up to 20 managed policies.

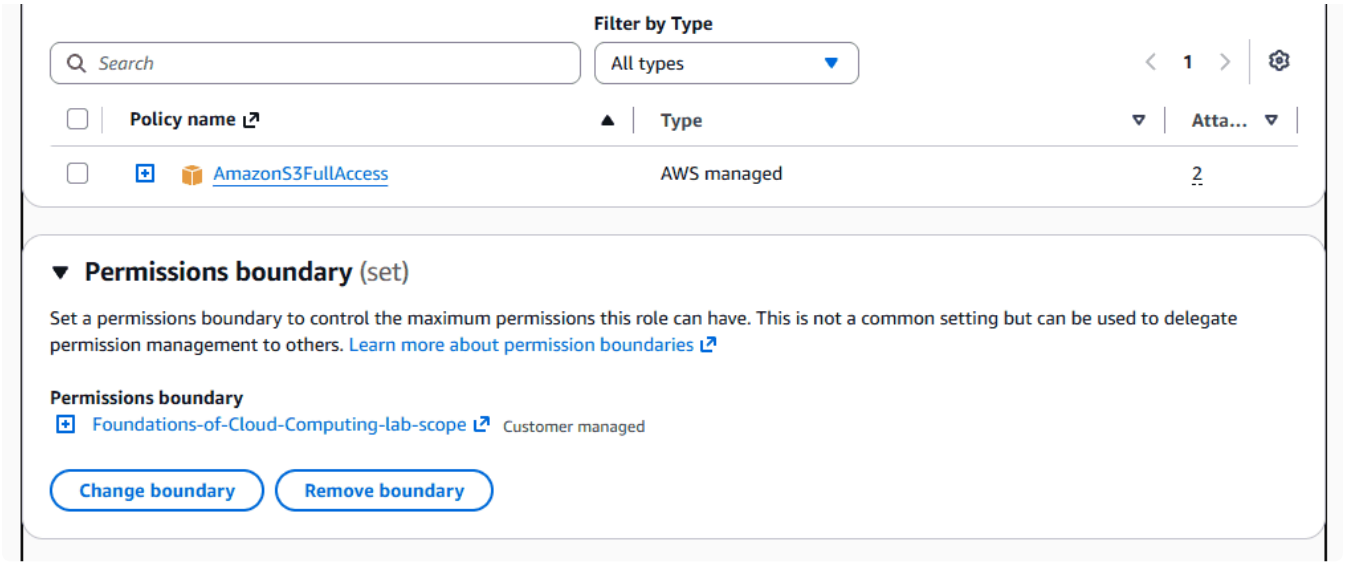


Рис. 5. IAM-роль `focc-ecs-task-execution-matiushko`: права ECS на pull образів із ECR, запис логів і читання секретів; permissions boundary курсу обмежує максимальні права.

Рис. 6. IAM-роль `focc-ecs-task-matiushko`: права самого застосунку на роботу з S3 (аватарки). Завдяки ролі у коді та env немає жодних ключів доступу.

Дані

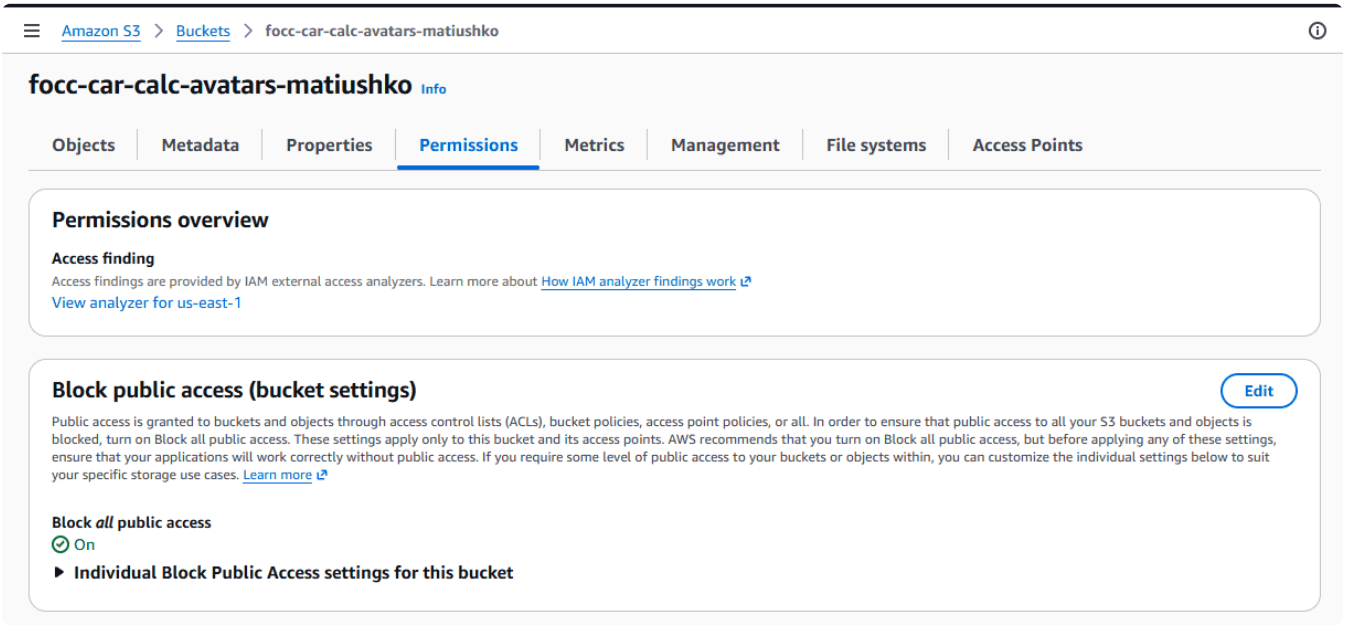


Рис. 7. S3-бакет `focc-car-calc-avatars-matiushko`: Block all public access увімкнено — файли віддає бекенд, публічних посилань на об'єкти немає.

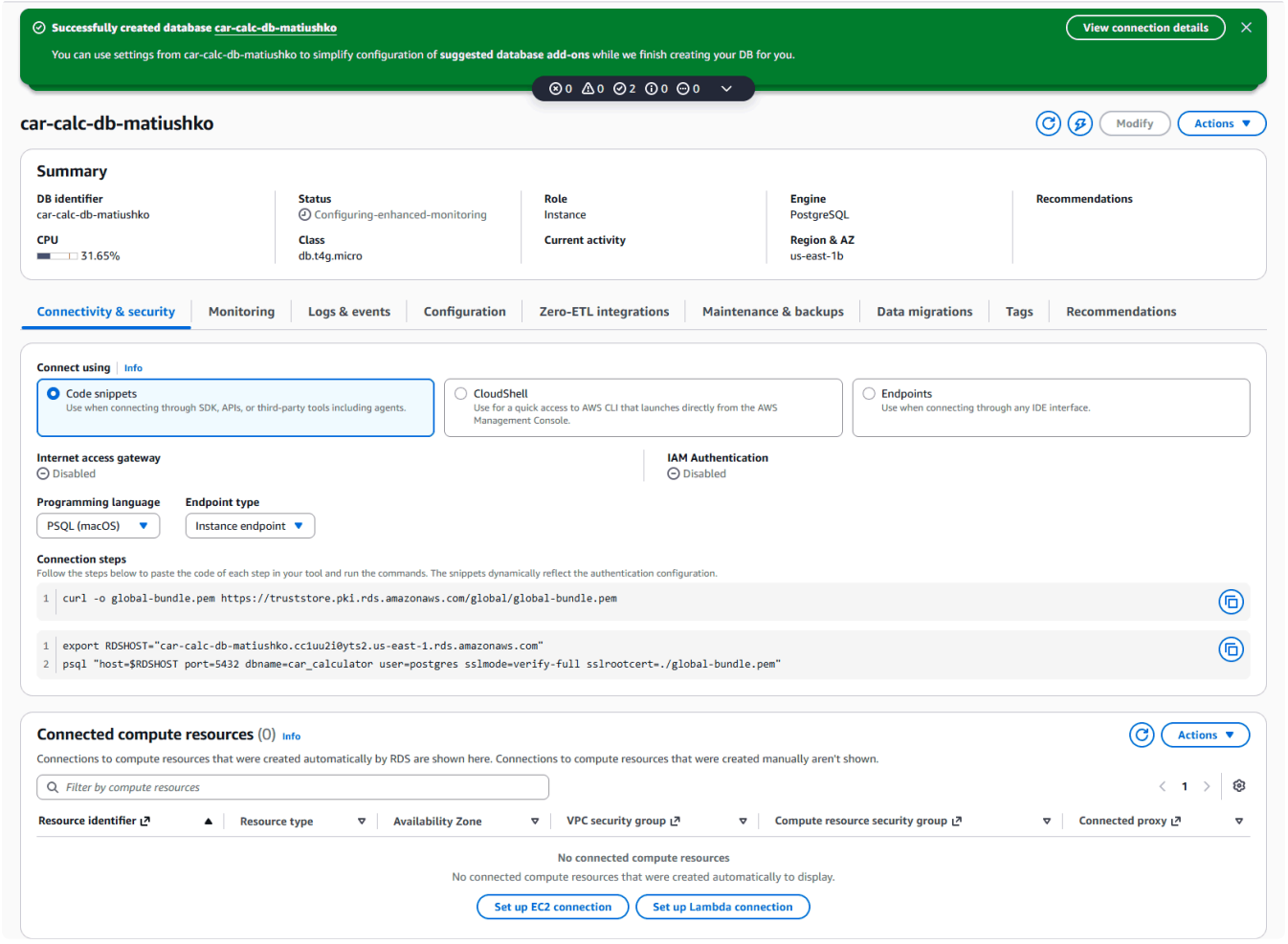


Рис. 8. RDS `car-calc-db-matiushko`, Connectivity & security: endpoint бази, Internet access gateway — Disabled (без публічного доступу).

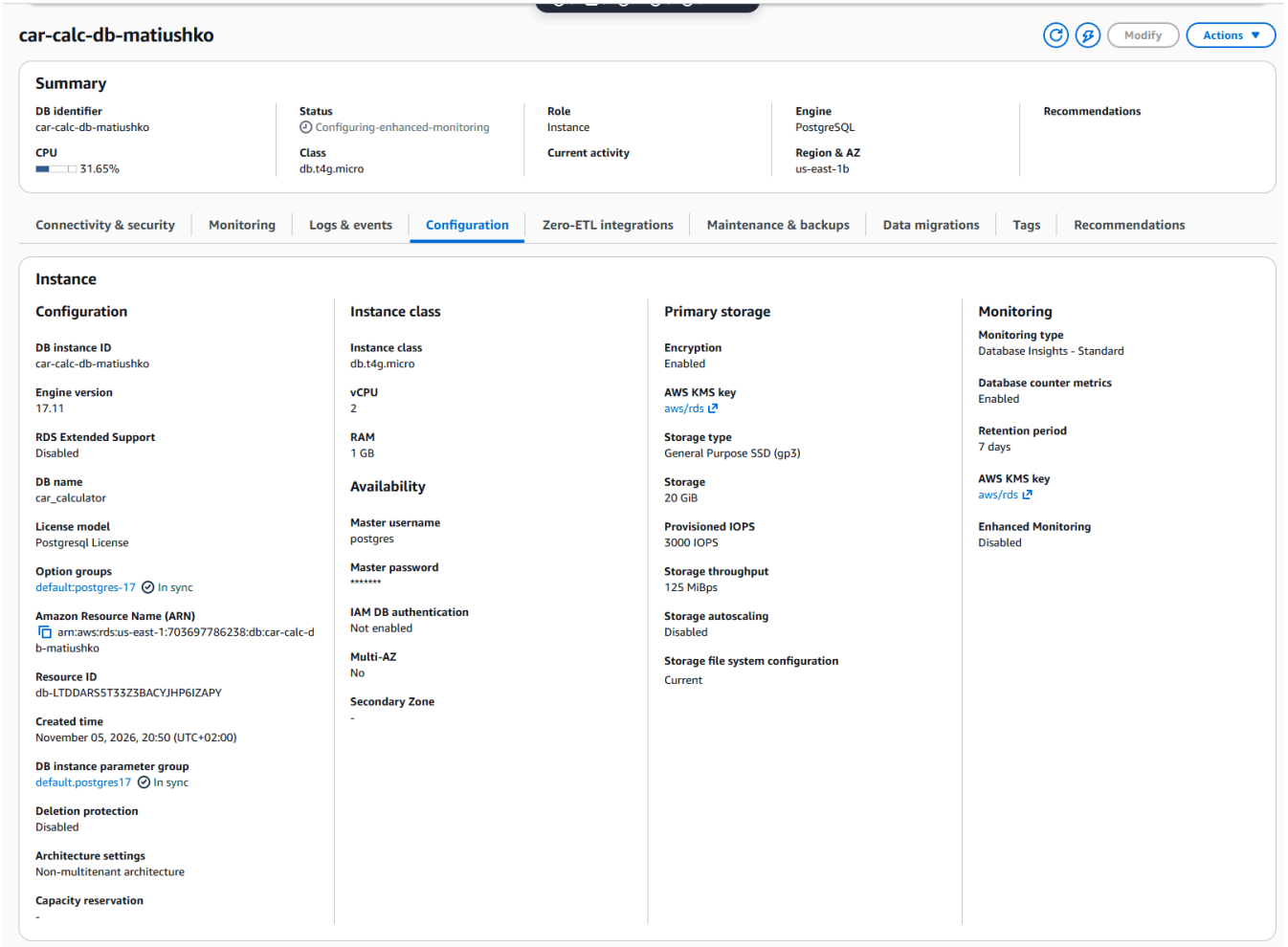


Рис. 9. Конфігурація RDS: PostgreSQL 17.11, db.t4g.micro, 20 GiB gp3, шифрування увімкнено, Multi-AZ — No, Enhanced Monitoring вимкнено (мінімальна вартість).

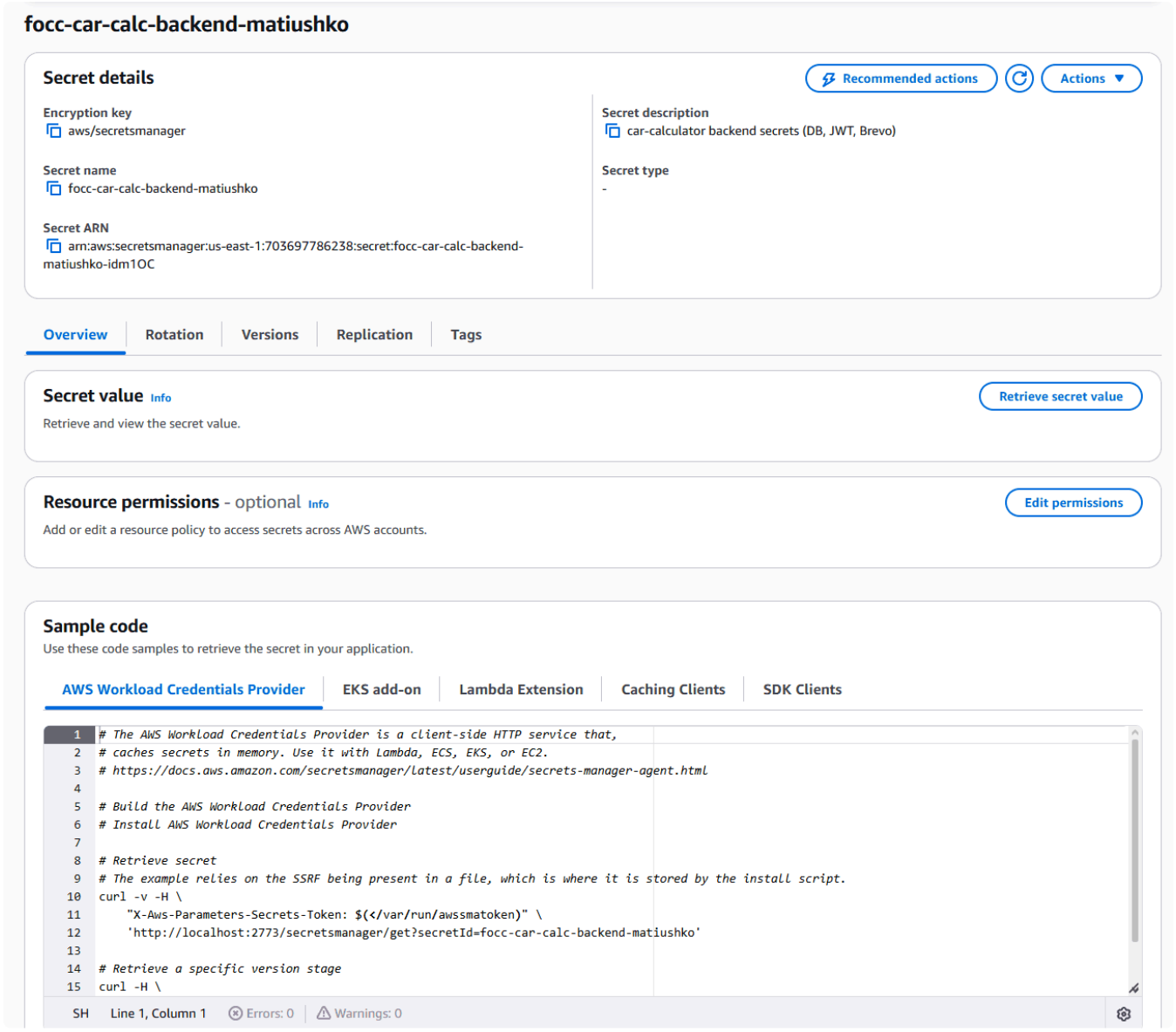


Рис. 10. Secrets Manager: секрет `focc-car-calc-backend-matiushko` з рядками підключення до БД, JWT-ключами та API-ключем; значення не розкриті.

Образи і сервіси

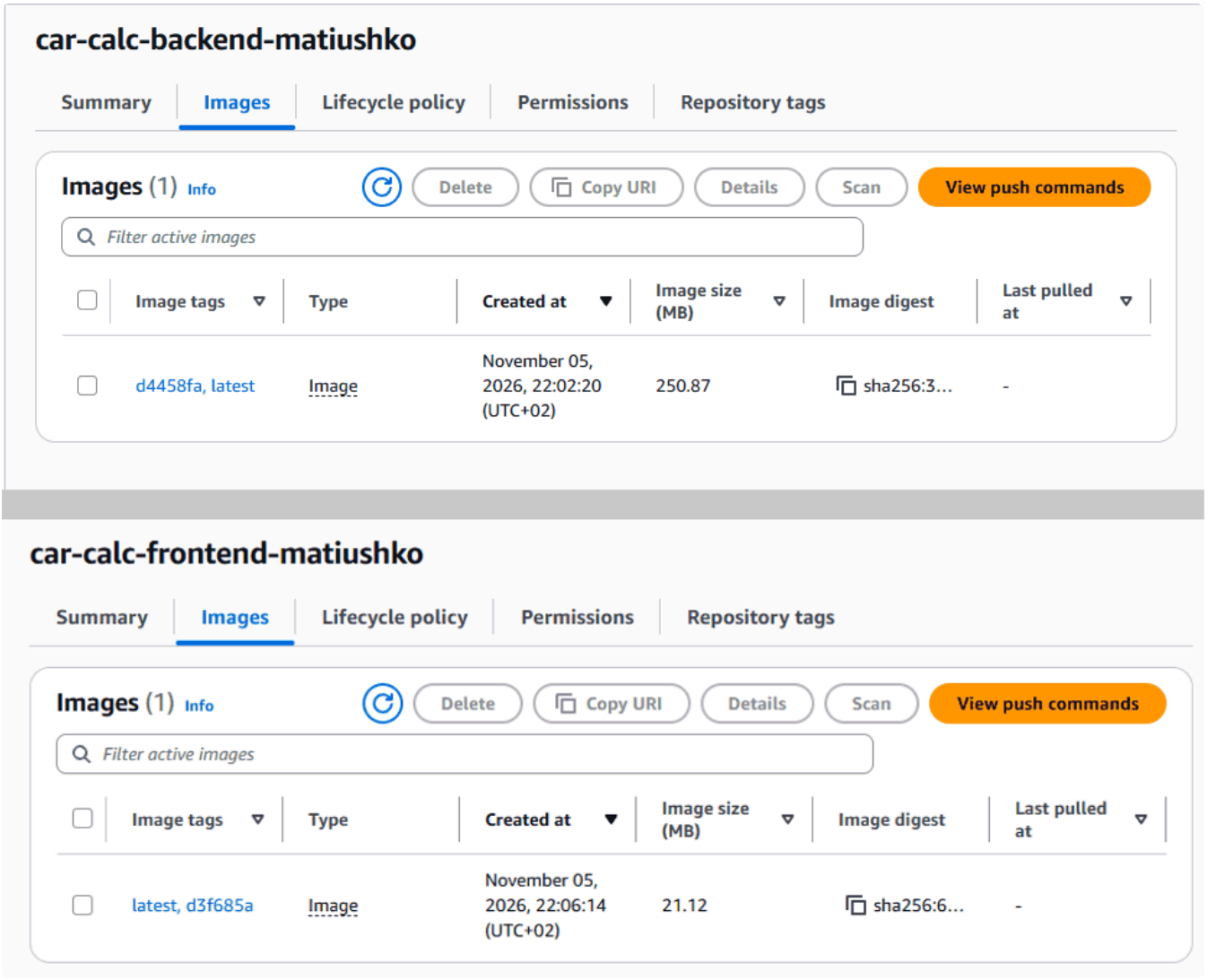


Рис. 11. ECR: образ бекенду з тегами `latest` і git-SHA (251 MB).

Рис. 12. ECR: образ фронтенду (nginx + статика React, 21 MB).

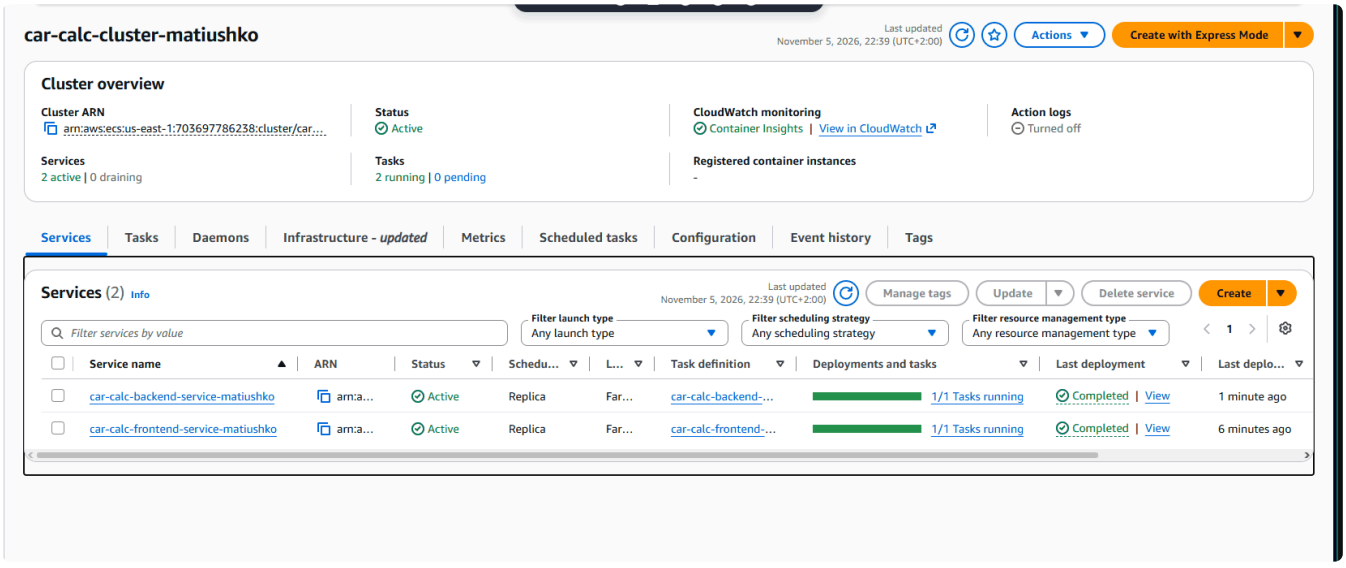


Рис. 13. ECS-кластер `car-calc-cluster-matiushko`: обидва сервіси Active, по 1/1 таску, деплой Completed.

Застосунок за URL ALB

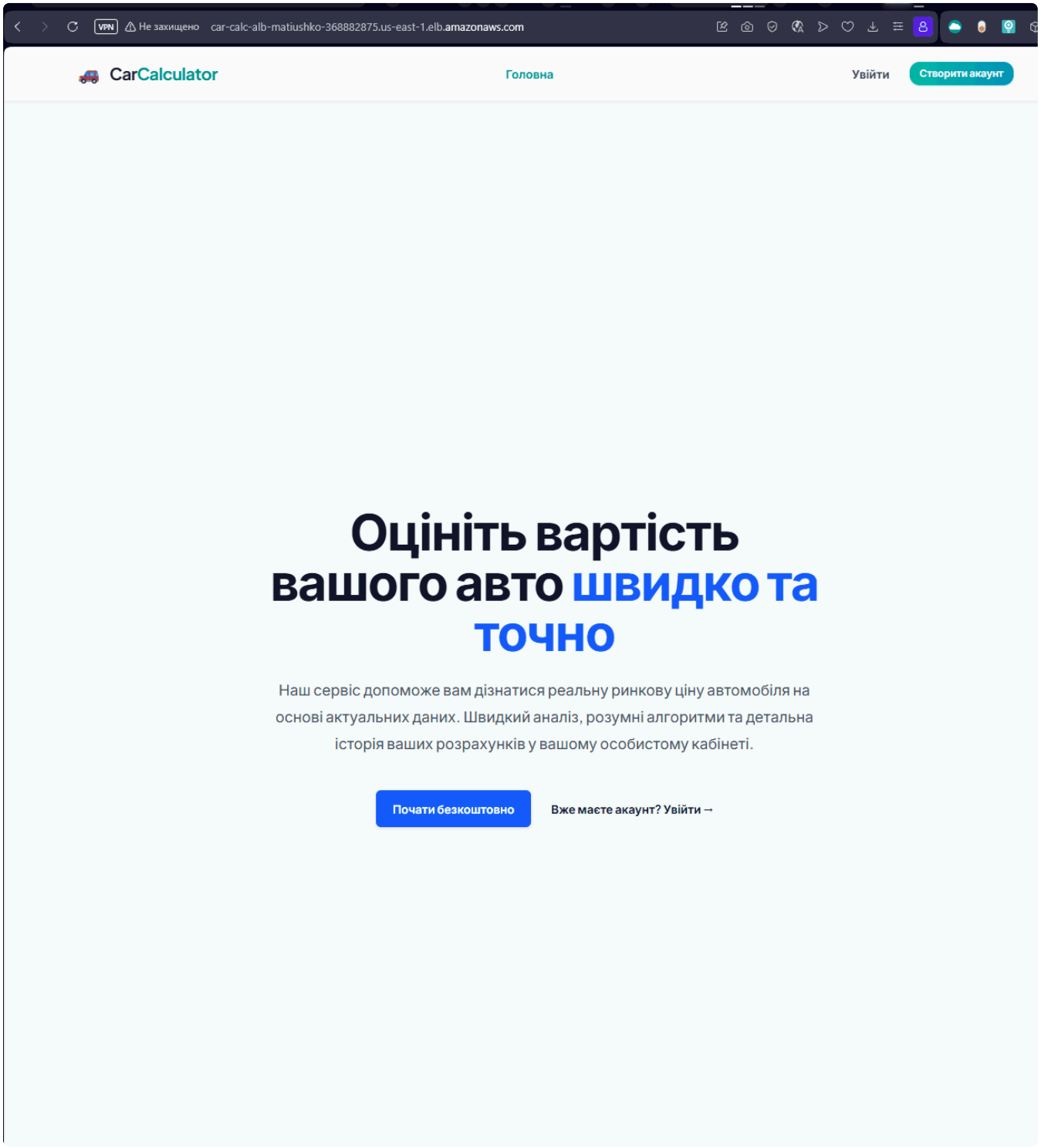


Рис. 14. Застосунок відкрито за DNS-ім'ям ALB — головна сторінка фронтенду.

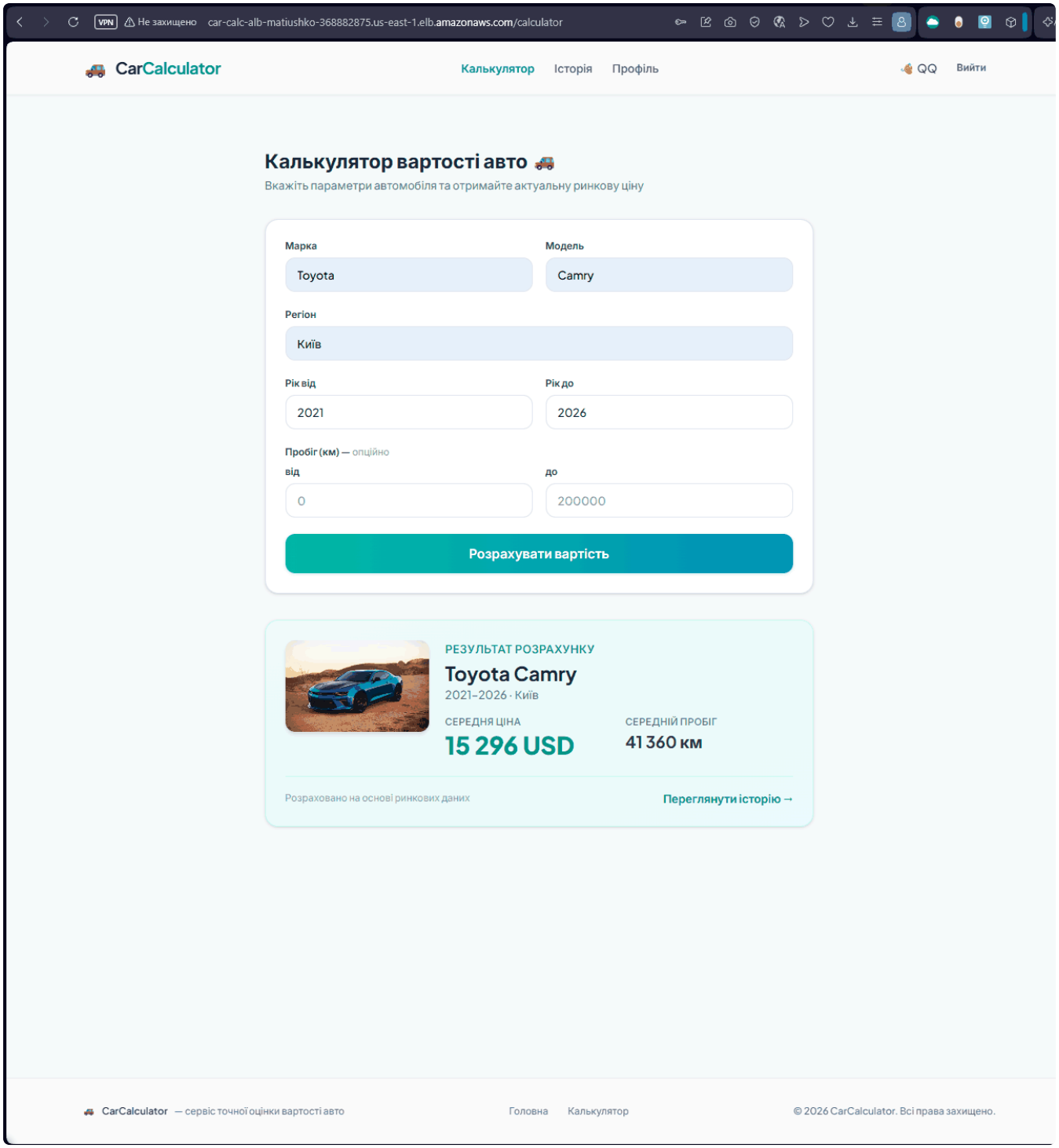


Рис. 15. Розрахунок ціни авто після входу: запит пройшов через ALB до бекенду, результат збережено в RDS.

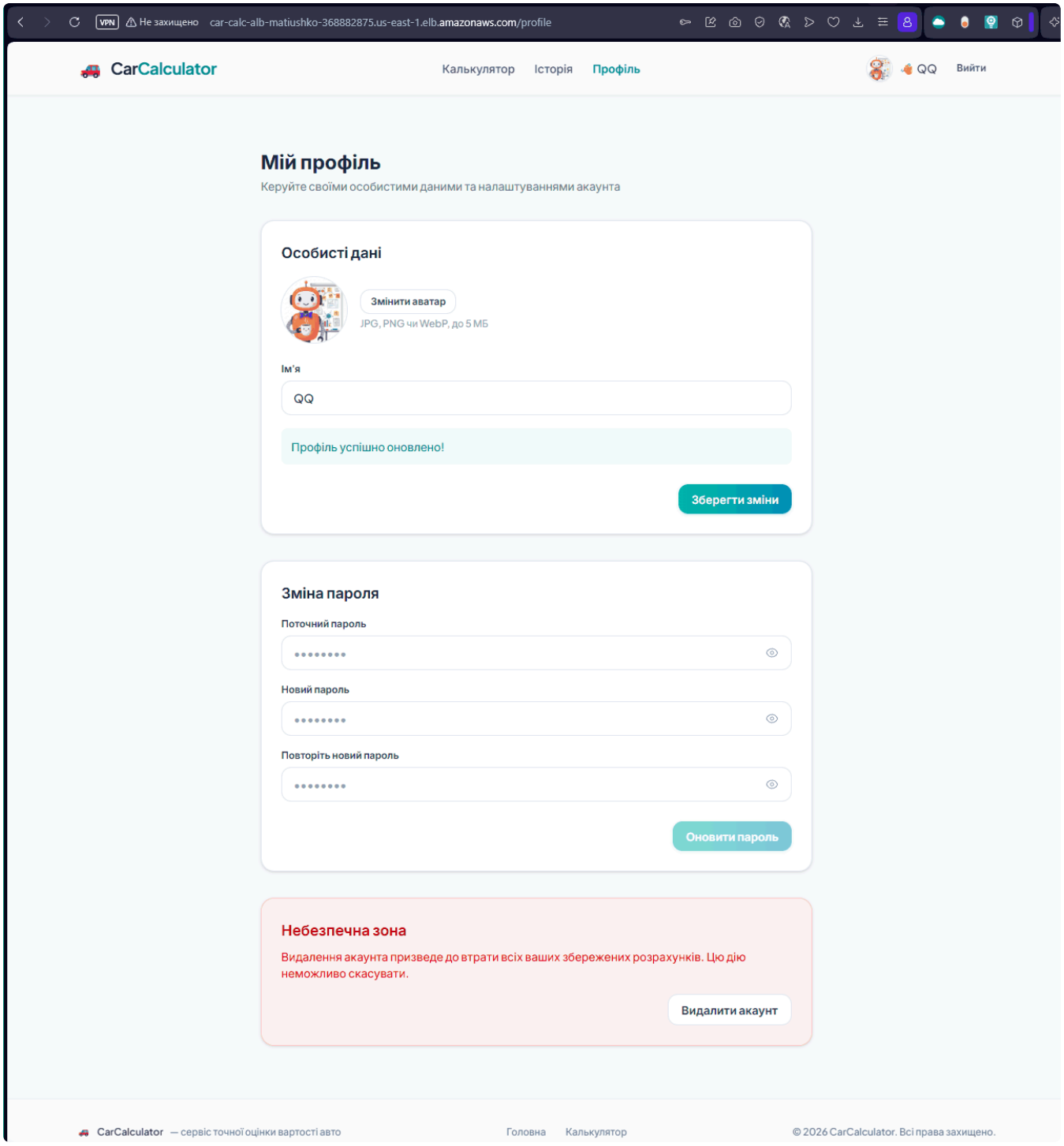


Рис. 16. Профіль користувача з аватаркою, завантаженою в S3.

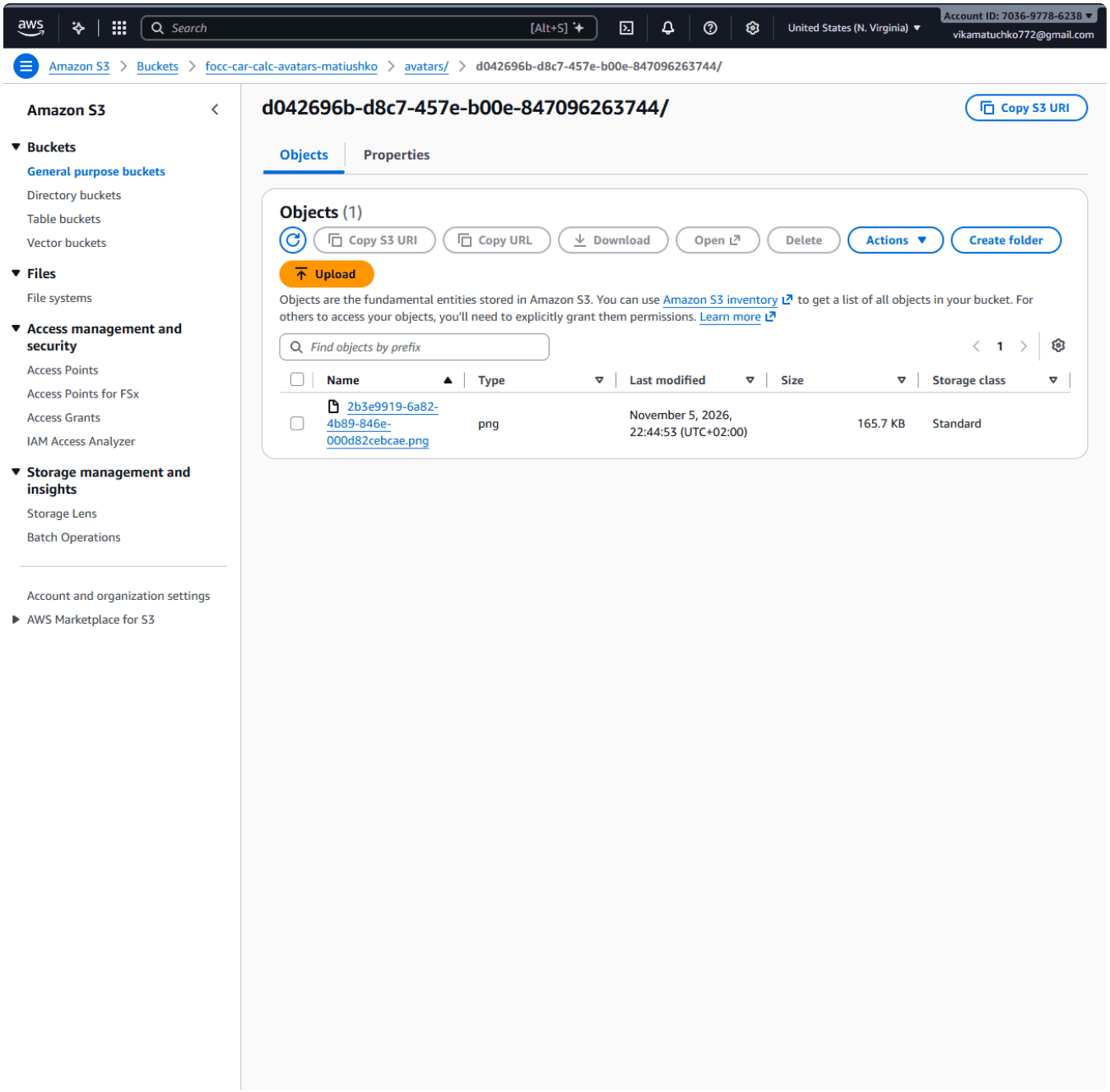


Рис. 17. Той самий файл у бакеті: `avatars/<userId>/<uuid>.png` — підтвердження, що бекенд пише в S3 через task role.

9.2. Балансування і автомасштабування

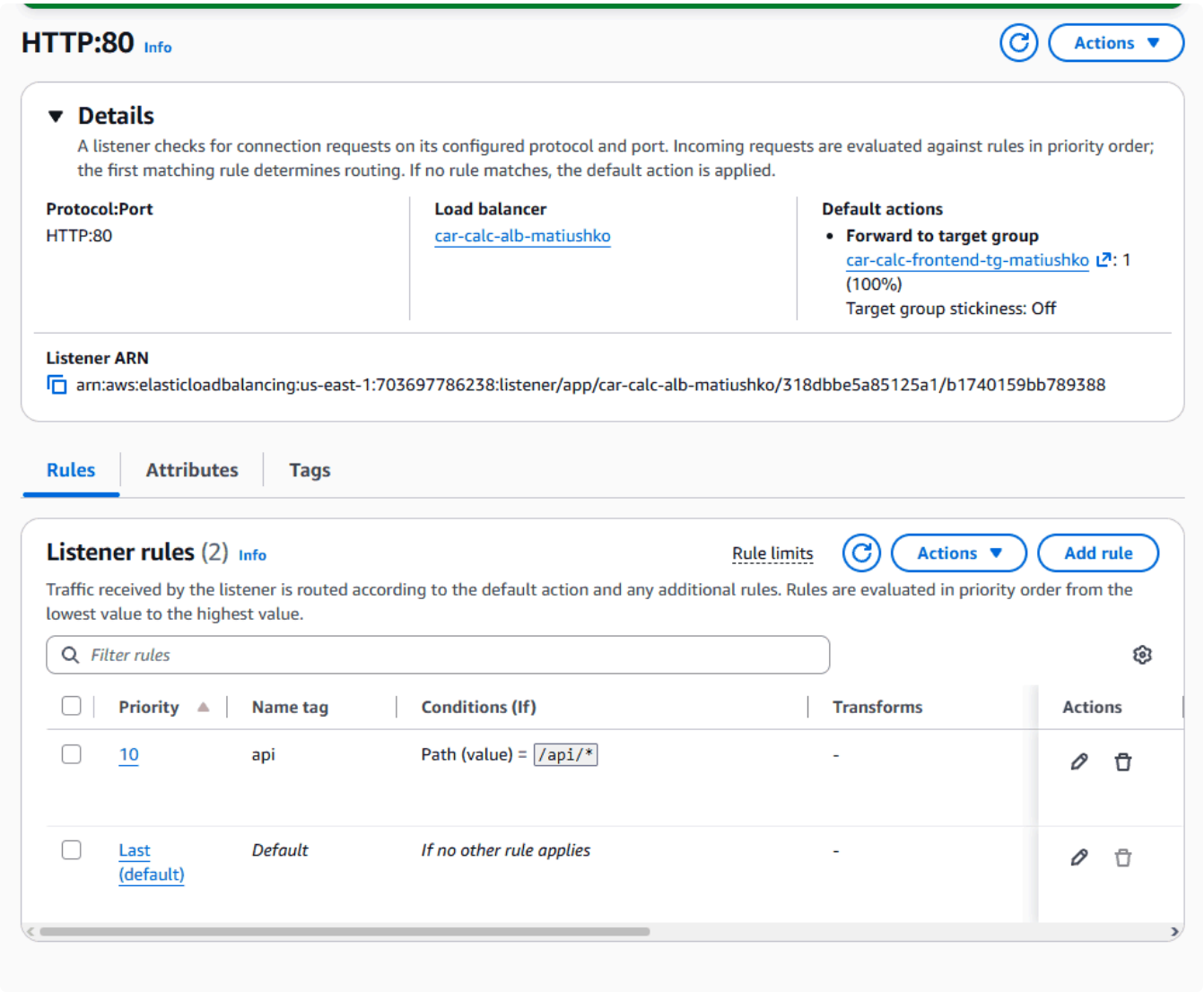


Рис. 18. Правила listener HTTP:80: запити `/api/*` (priority 10) йдуть на backend target group, усе інше (default) — на frontend.

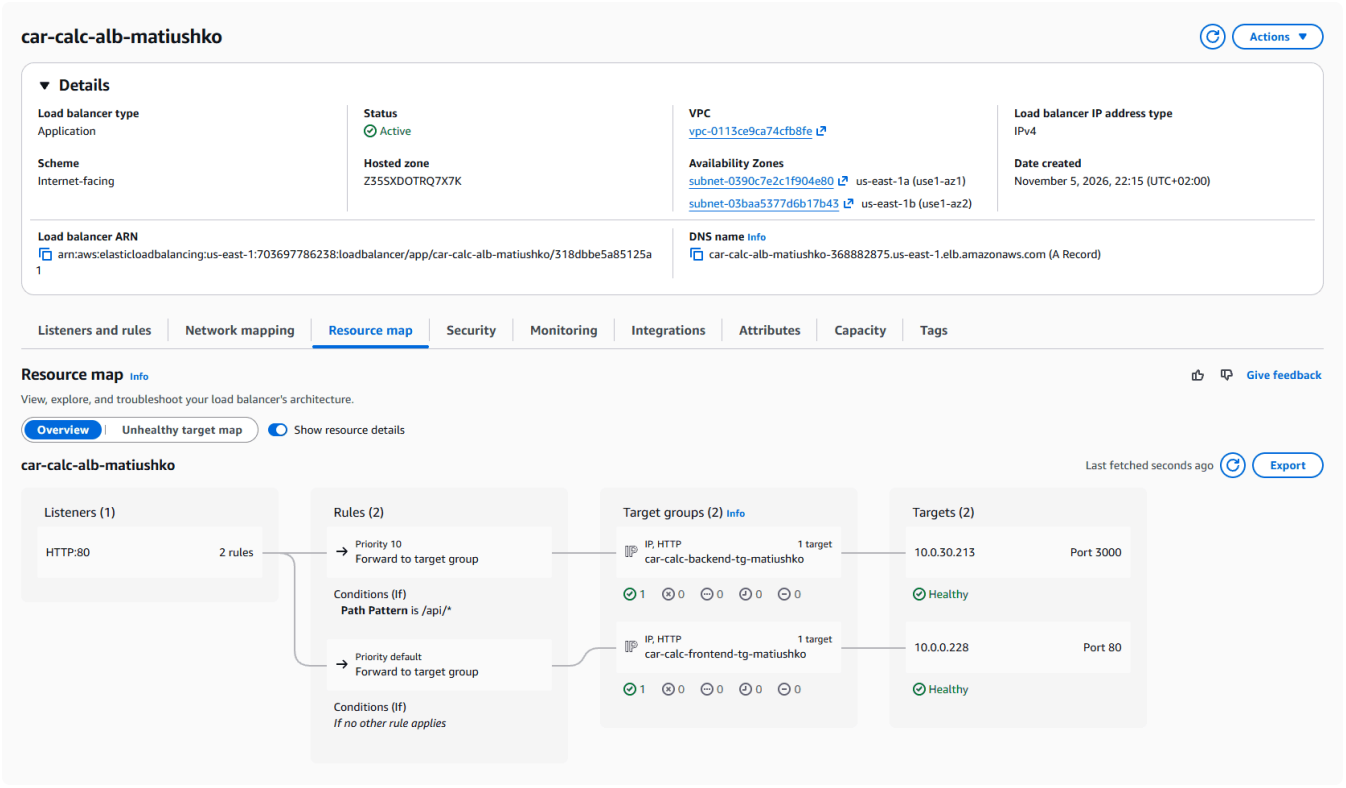


Рис. 19. Resource map ALB: listener → 2 правила → 2 target groups → таргети тасків у двох AZ, обидва Healthy.

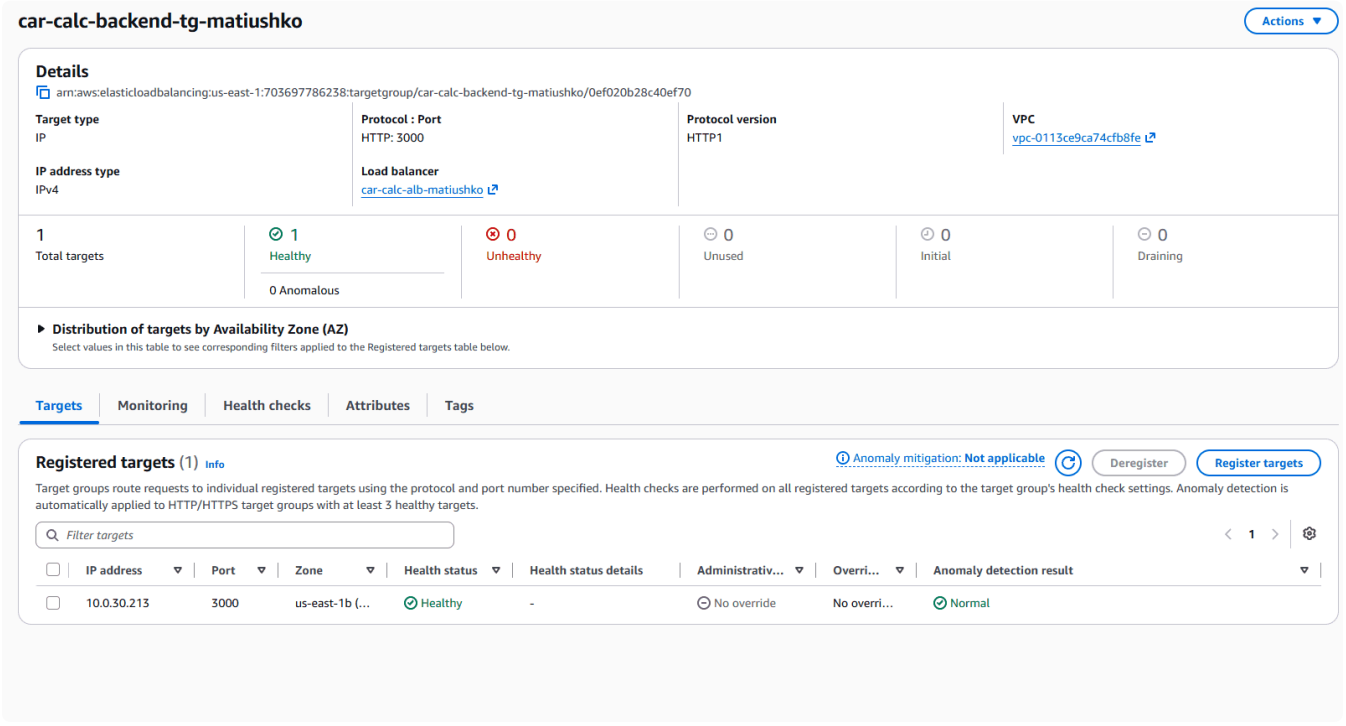


Рис. 20. Target group бекенду: таск зареєстрований за IP і проходить health check `/api/health`.

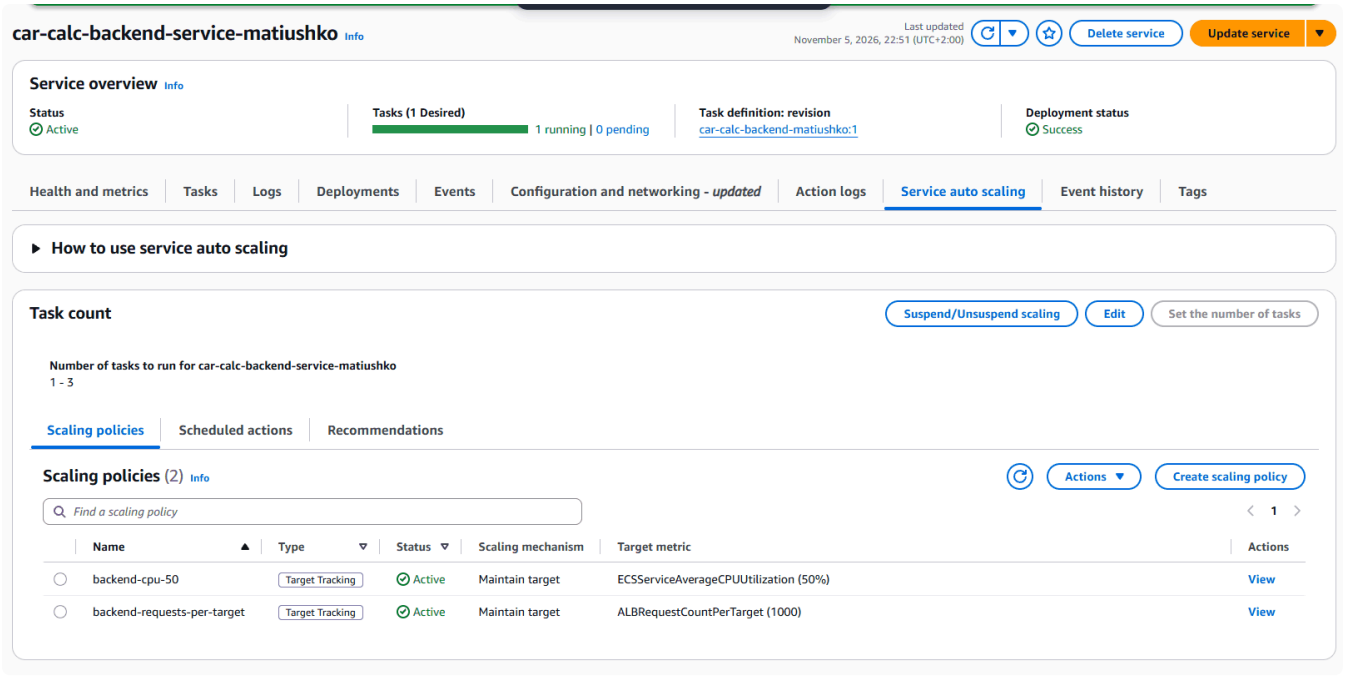


Рис. 21. Service auto scaling бекенду: діапазон 1–3 таски, дві target-tracking політики — CPU 50 % і 1000 запитів/хв на таск.

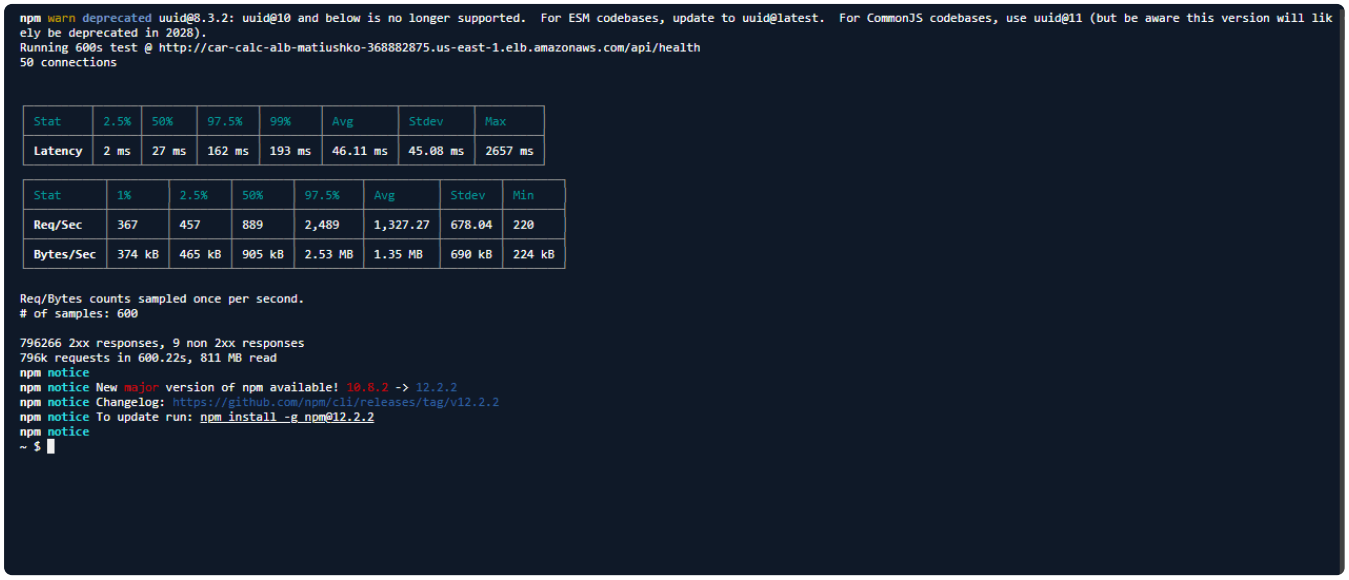


Рис. 22. Навантажувальний тест 1 (autocannon, 50 з'єднань, 10 хв): 796 тис. запитів, в середньому 1 327 req/s, медіана 27 ms.

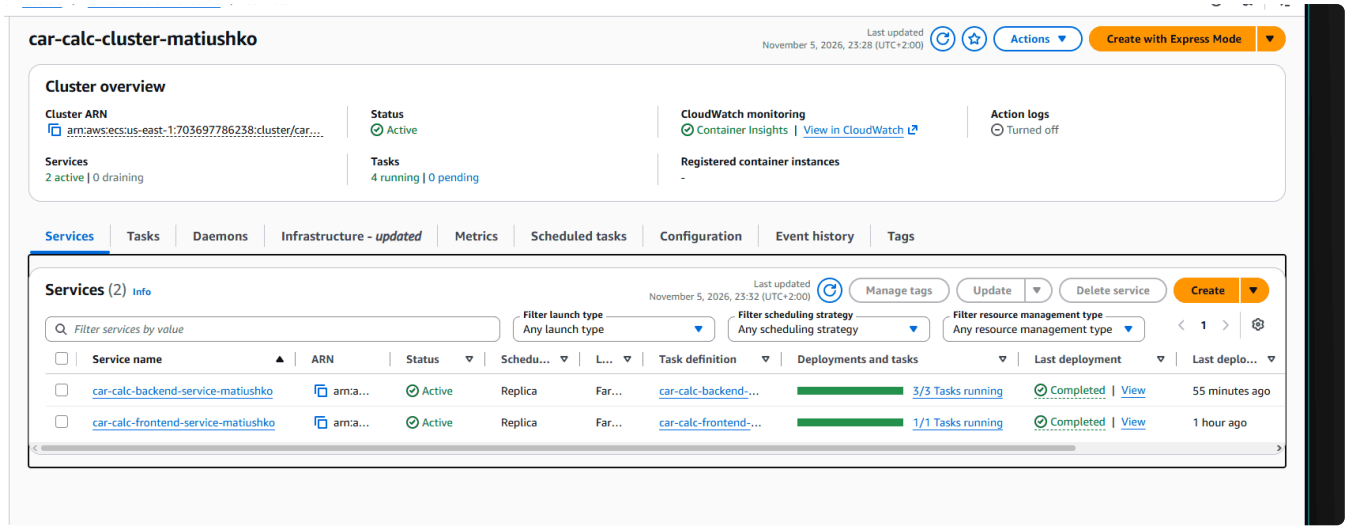


Рис. 23. Під час тесту бекенд масштабувався до 3/3 тасків (у кластері 4 running разом із фронтендом).

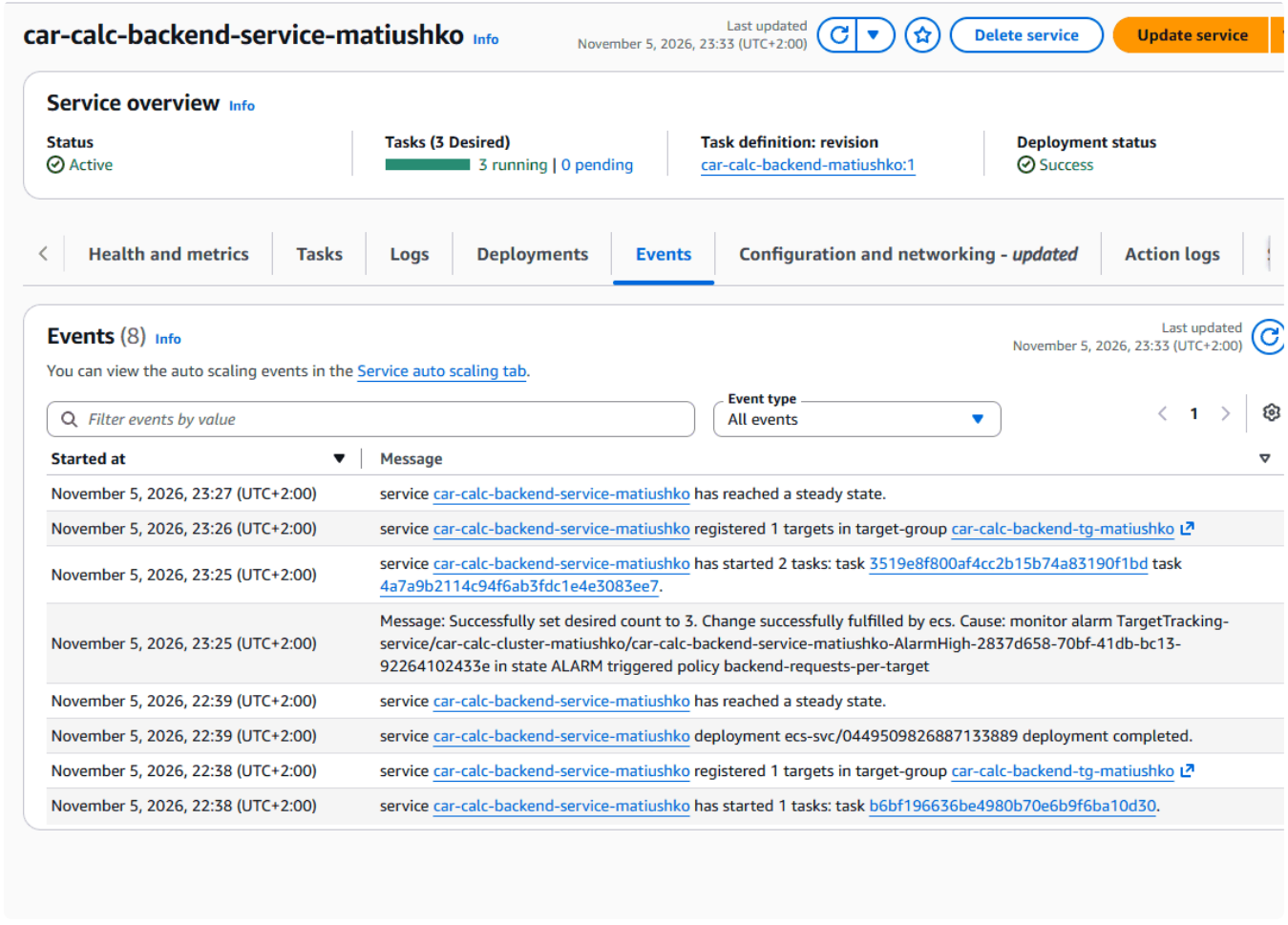


Рис. 24. Events сервісу (scale-out): AlarmHigh політики `backend-requests-per-target` → desired count 3 → запущено 2 таски → зареєстровані в target group → steady state.

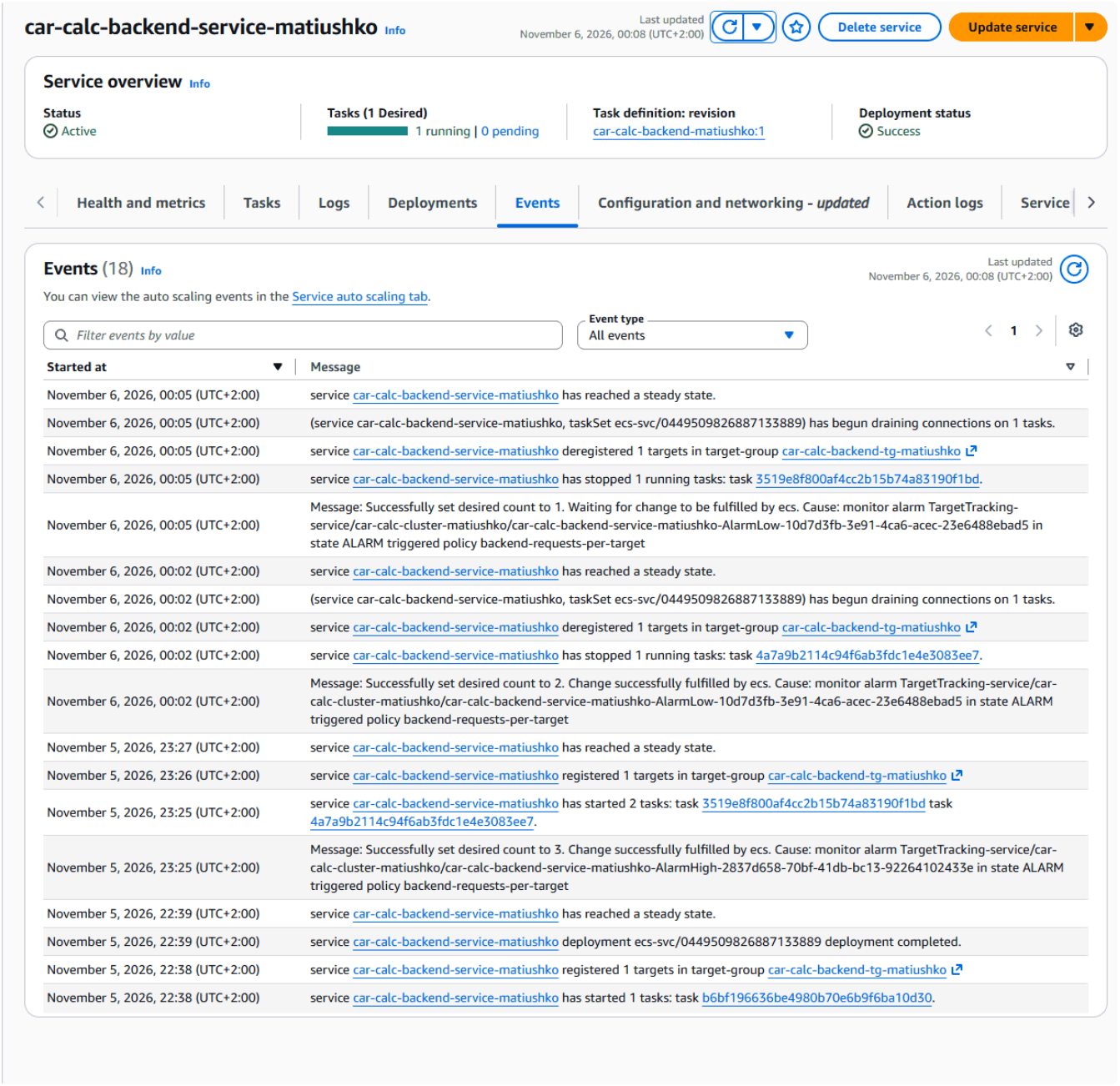


Рис. 25. Events сервісу (scale-in): після навантаження AlarmLow спершу знизив desired count до 2 (00:02), потім до 1 (00:05). Перед зупинкою таск спершу дереєструється в ALB і дочекується завершення з'єднань (draining) — запити не втрачаються.

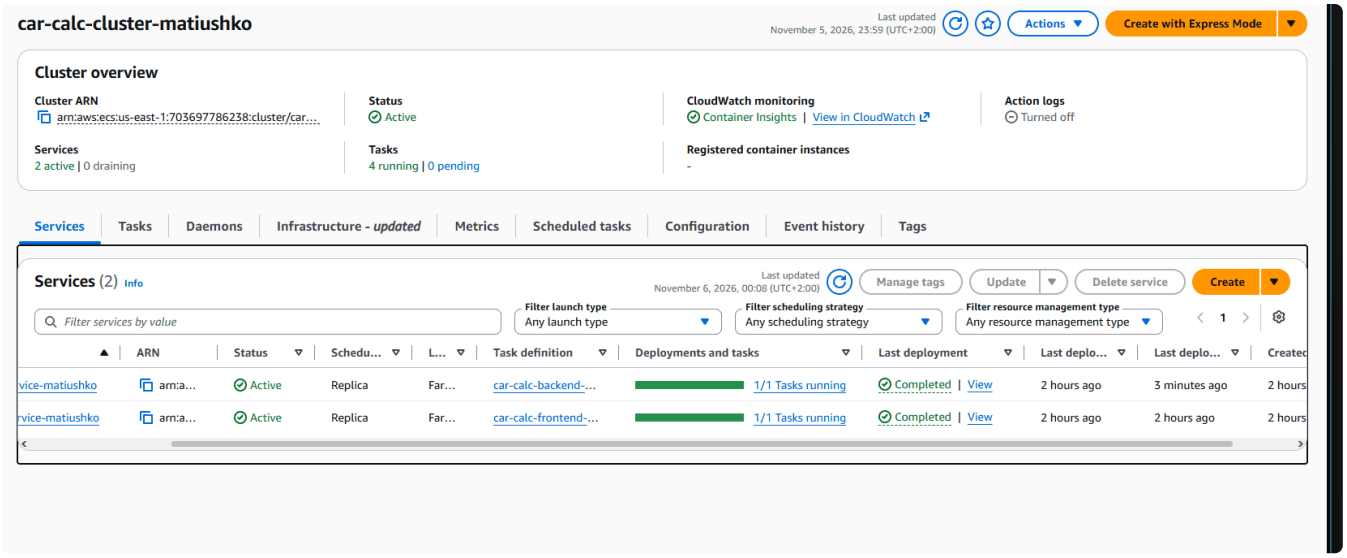


Рис. 26. Після scale-in обидва сервіси знову працюють на одному таску — додаткові ресурси працювали (і коштували) лише під час піку.

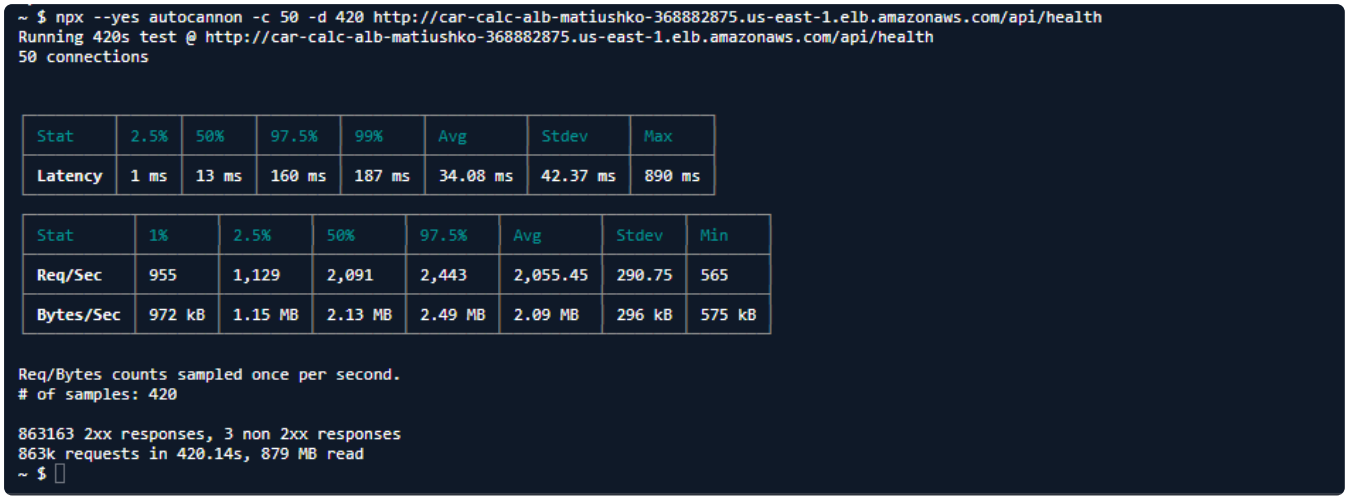


Рис. 27. Тест 2 (7 хв, таски вже розгорнуті): 863 тис. запитів, 2 055 req/s, медіана 13 ms — ефект горизонтального масштабування.

9.3. Моніторинг CloudWatch

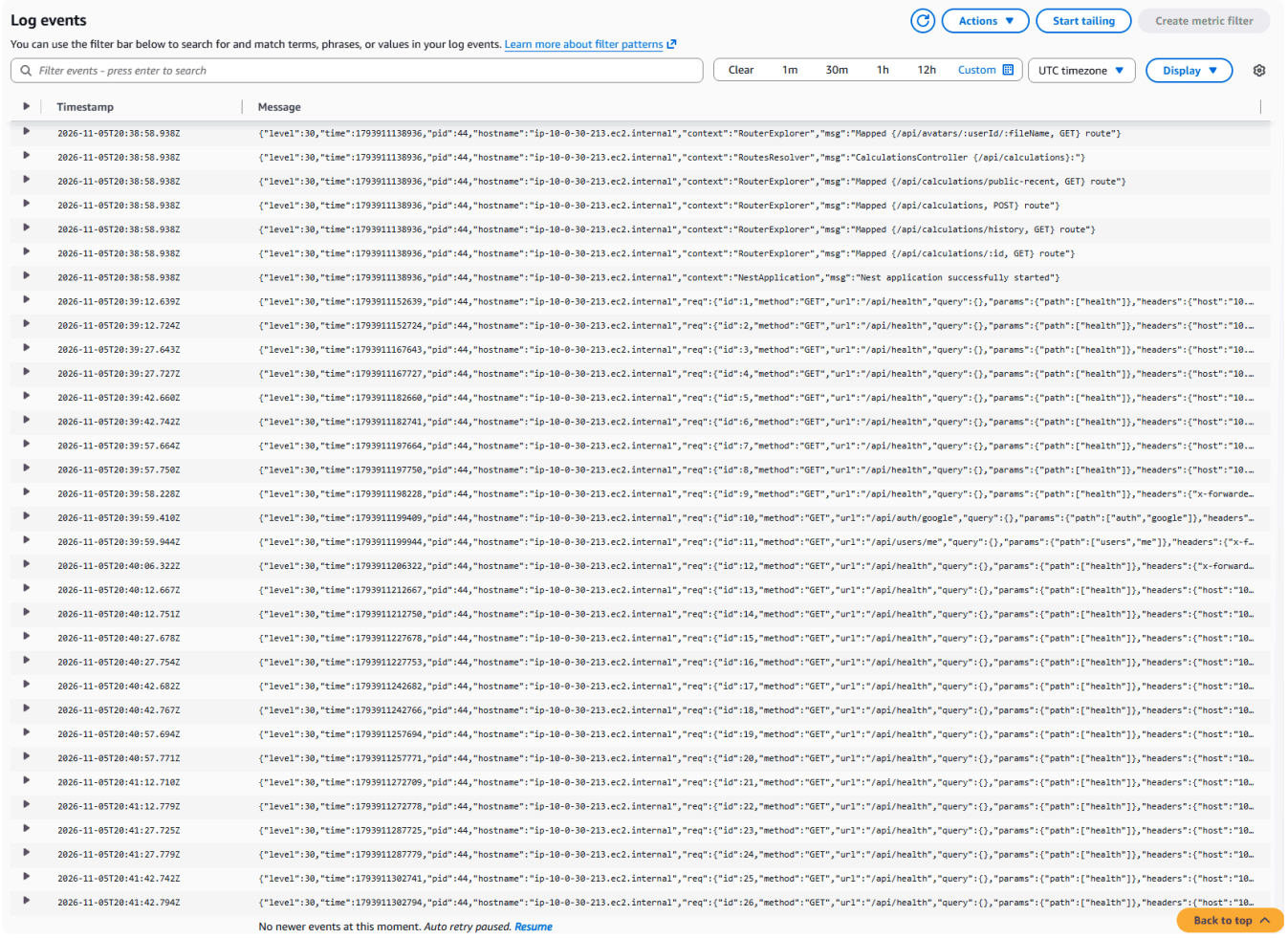


Рис. 28. CloudWatch Logs бекенду: старт NestJS («Nest application successfully started», змаплені маршрути `/api/...`), далі — регулярні health checks від ALB у форматі JSON.

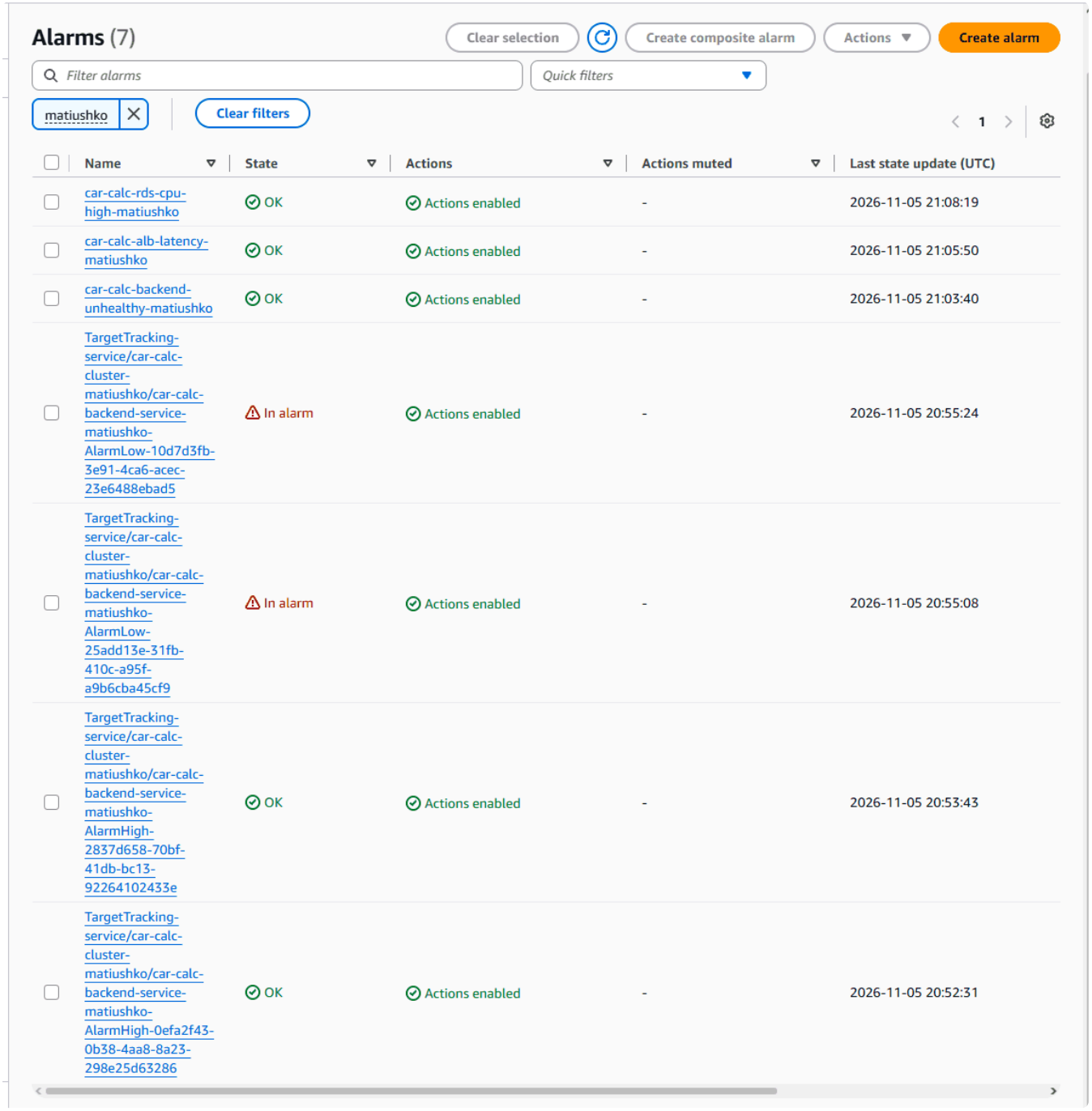


Рис. 29. CloudWatch Alarms (фільтр `matiushko`): 3 власні алярми зі сповіщенням у SNS (unhealthy hosts, latency, RDS CPU) у стані OK і 4 алярми автоскейлінгу.

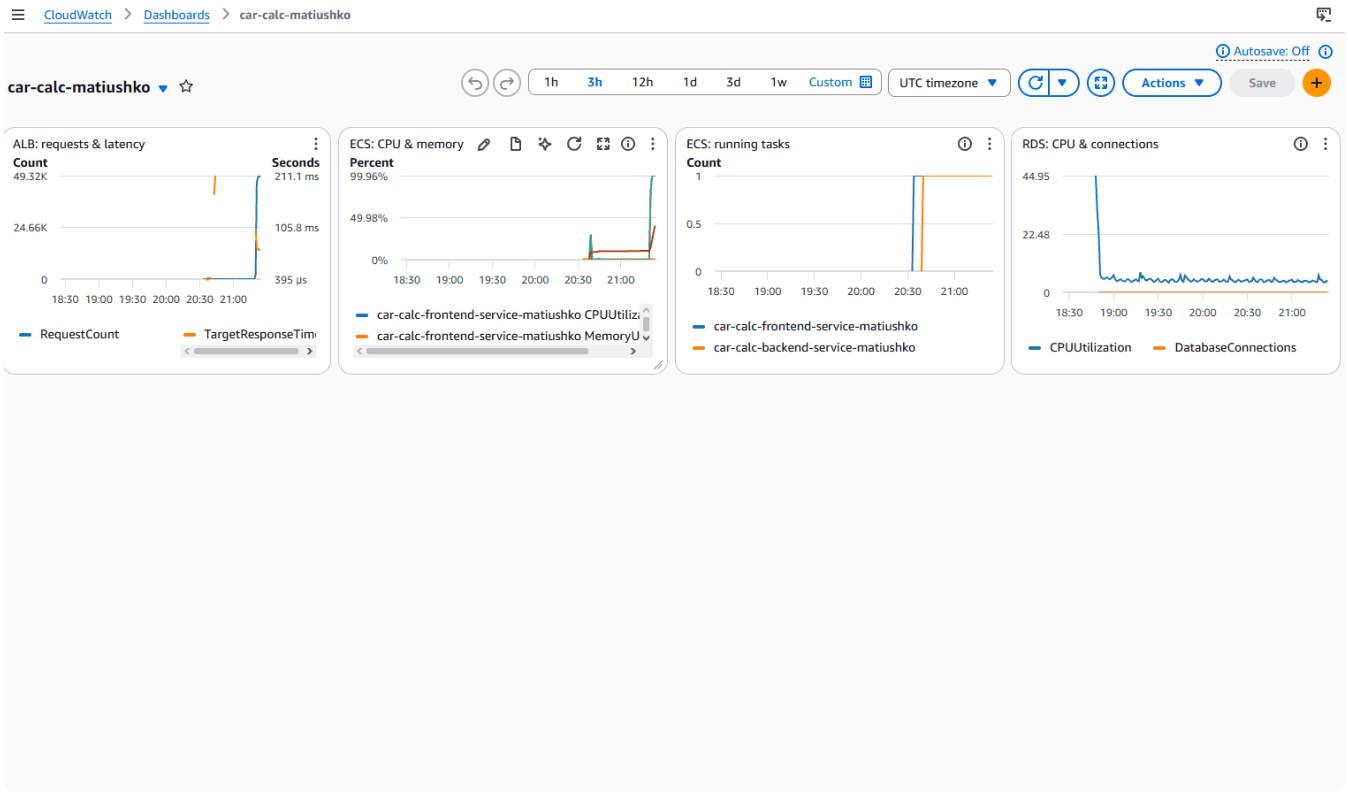


Рис. 30. Дашборд `car-calc-matiushko` під час тесту 1: різке зростання RequestCount і CPU бекенду до ~100 %.

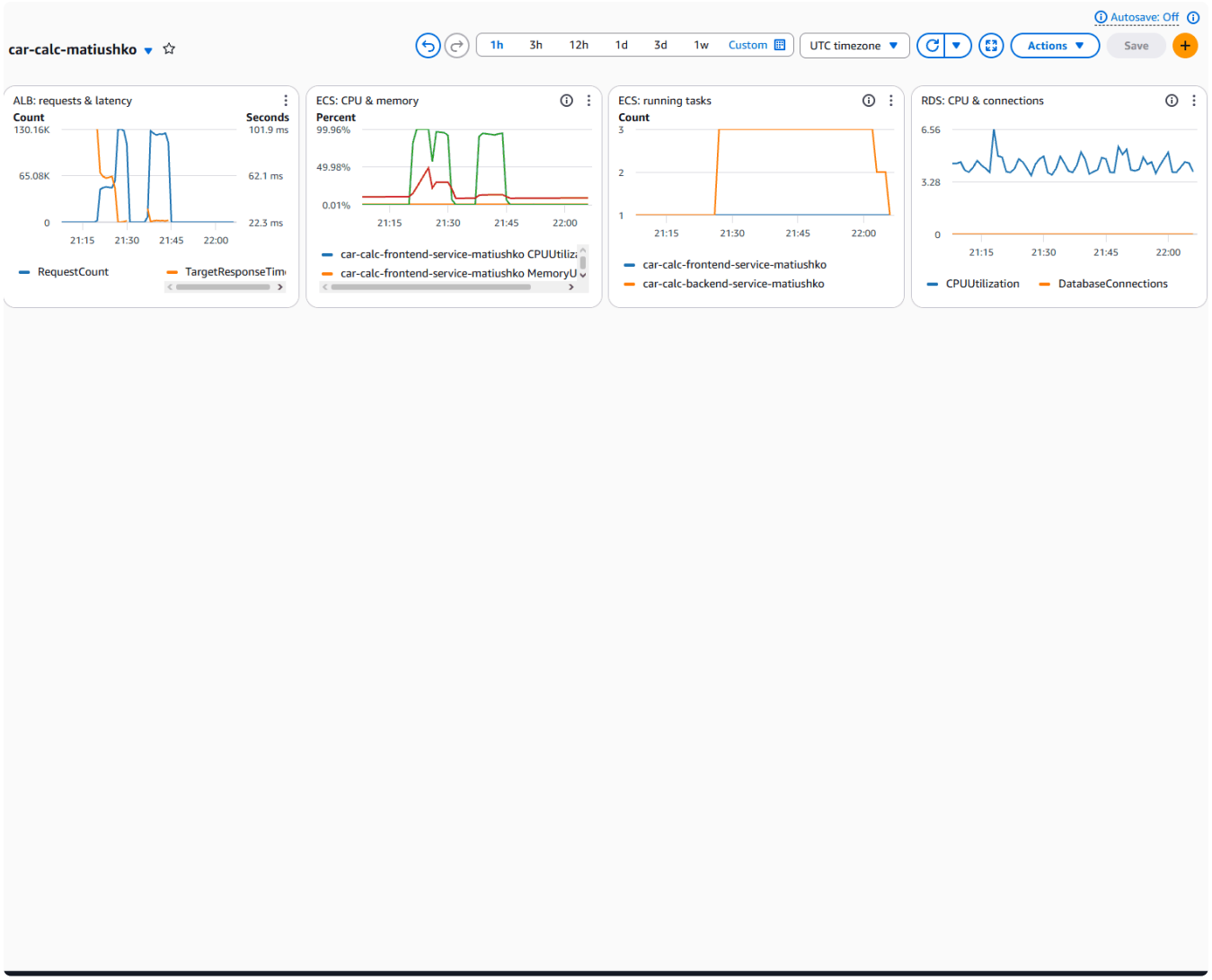


Рис. 31. Той самий дашборд за годину — повний цикл: два піки запитів (тести 1 і 2), CPU бекенду на піках, кількість тасків 1 → 3 → 2 → 1, база не навантажена (health-ендпоінт не звертається до БД).

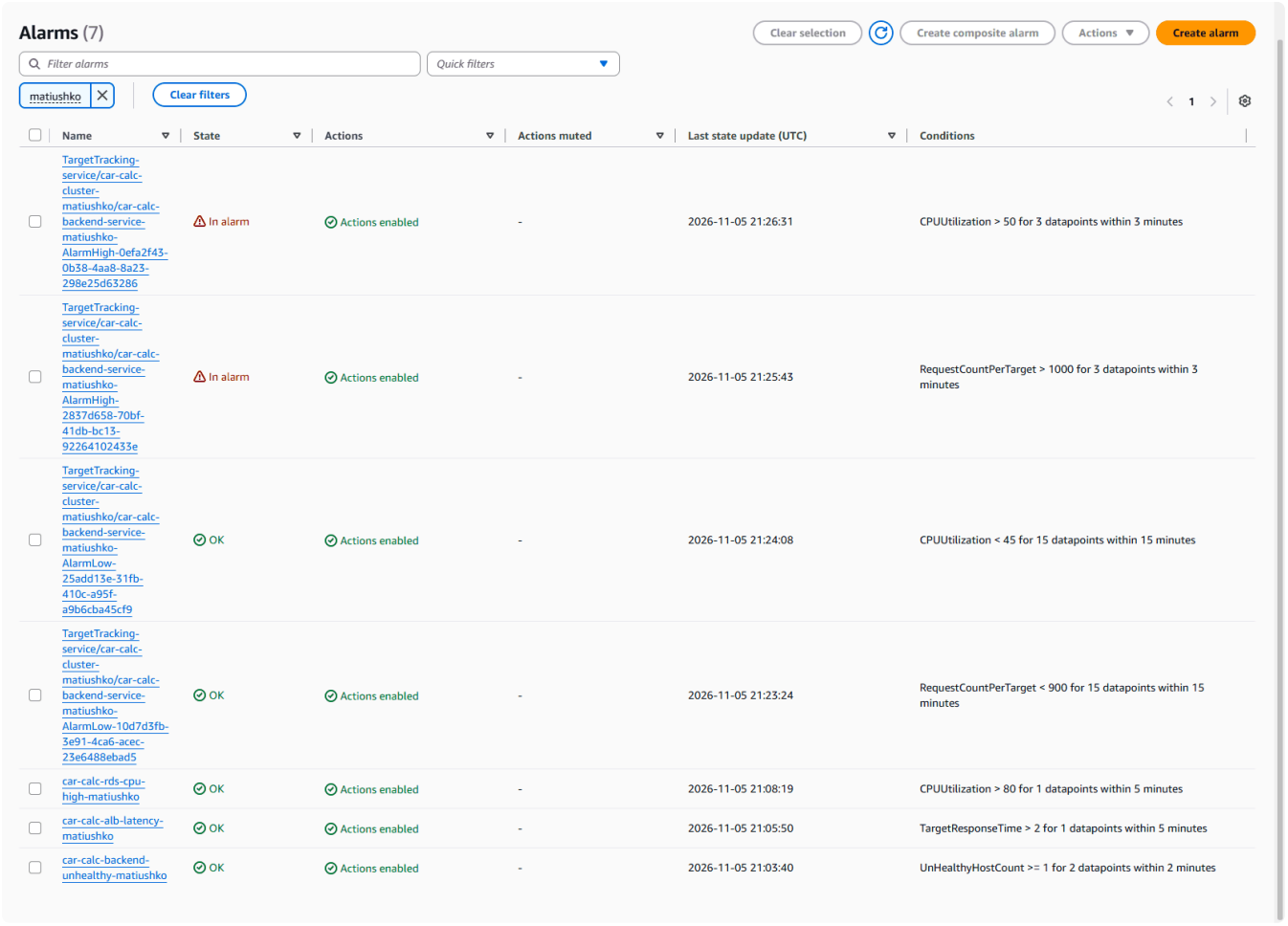


Рис. 32. Під час тесту обидва AlarmHigh автоскейлінгу перейшли в ALARM: CPU > 50 % і RequestCountPerTarget > 1000 протягом 3 хв.

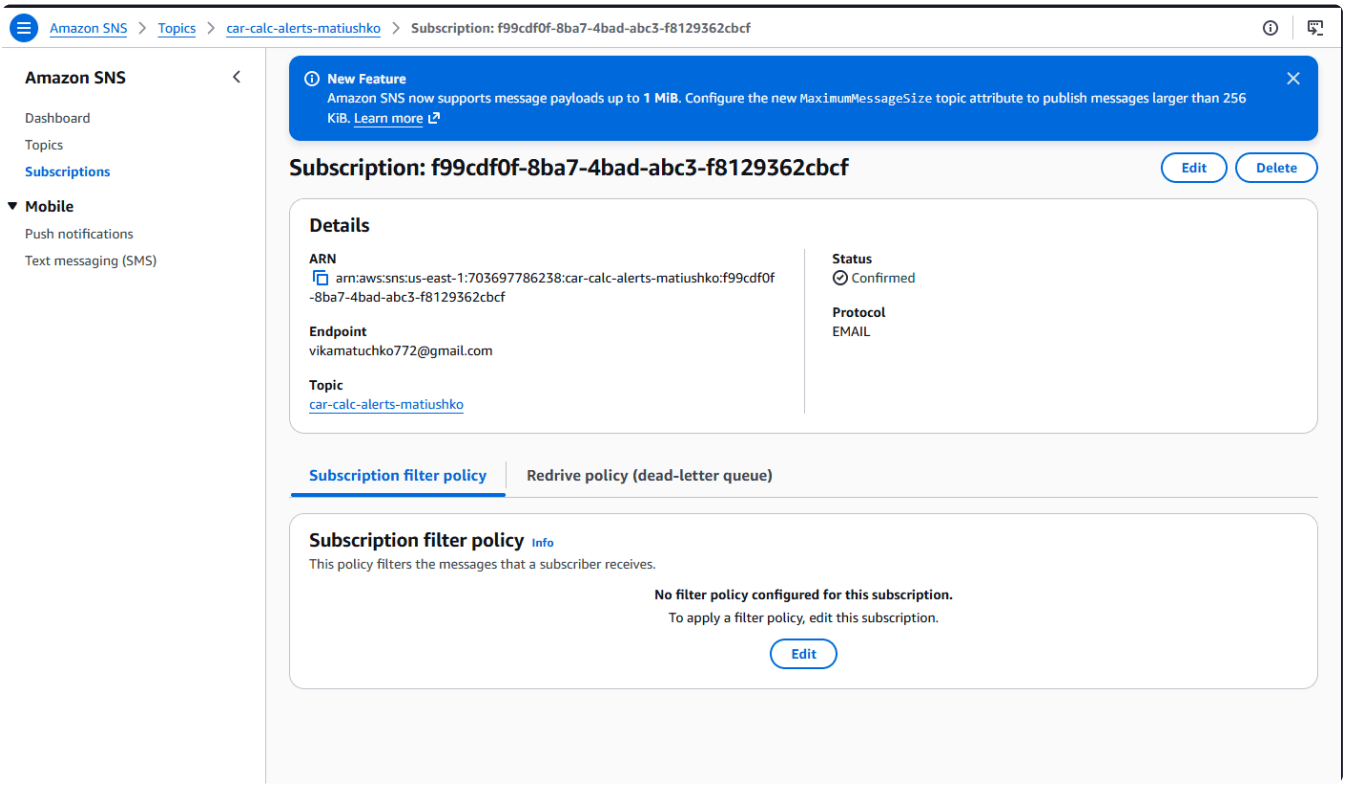


Рис. 33. SNS-топік `car-calc-alerts-matiushko`: email-підписку підтверджено (Confirmed).

1 з 6 028

ALARM: "car-calc-alb-latency-matiushko" in US East (N. Virginia)

Вхідні ×

car-calc-alerts-matiushko <no-reply@sns.amazonaws.com> 23:38 (2 хвилини тому)
кому мені

You are receiving this email because your Amazon CloudWatch Alarm "car-calc-alb-latency-matiushko" in the US East (N. Virginia) region has entered the ALARM state, because "Threshold Crossed: 1 out of the last 1 datapoints [0.02563188996095768 (05/11/26 21:26:00)] was greater than the threshold (0.02) (minimum 1 datapoint for OK -> ALARM transition)." at "Thursday 05 November, 2026 21:38:53 UTC".

View this alarm in the AWS Management Console:
https://us-east-1.console.aws.amazon.com/cloudwatch/deeplink.js?region=us-east-1#alarmsV2:alarm/car-calc-alb-latency-matiushko

Alarm Details:
- Name: car-calc-alb-latency-matiushko
- Description: ALB target response time > 2 s
- State Change: OK -> ALARM
- Reason for State Change: Threshold Crossed: 1 out of the last 1 datapoints [0.02563188996095768 (05/11/26 21:26:00)] was greater than the threshold (0.02) (minimum 1 datapoint for OK -> ALARM transition).
- Timestamp: Thursday 05 November, 2026 21:38:53 UTC
- AWS Account: 703697786238
- Alarm Arn: arn:aws:cloudwatch:us-east-1:703697786238:alarm:car-calc-alb-latency-matiushko

Threshold:
- The alarm is in the ALARM state when the metric is GreaterThanThreshold 0.02 for at least 1 of the last 1 period(s) of 300 seconds.

Monitored Metric:
- MetricNamespace: AWS/ApplicationELB
- MetricName: TargetResponseTime
- Dimensions: [LoadBalancer = app/car-calc-alb-matiushko/318dbbe5a85125a1]
- Period: 300 seconds
- Statistic: Average
- Unit: not specified
- TreatMissingData: notBreaching

State Change Actions:
- OK: [arn:aws:sns:us-east-1:703697786238:car-calc-alerts-matiushko]
- ALARM: [arn:aws:sns:us-east-1:703697786238:car-calc-alerts-matiushko]
- INSUFFICIENT_DATA:

--
If you wish to stop receiving notifications from this topic, please click or visit the link below to unsubscribe:
https://sns.us-east-1.amazonaws.com/unsubscribe-v2.html?SubscriptionArn=arn:aws:sns:us-east-1:703697786238:car-calc-alerts-matiushko:f99cdf0f-8ba7-4bad-abc3-f8129362cbcf&Endpoint=vikamatuchko772@gmail.com

Please do not reply directly to this email. If you have any questions or comments regarding this email, please contact us at https://aws.amazon.com/support

Відповісти Переслати

OK: "car-calc-alb-latency-matiushko" in US East (N. Virginia)

Вхідні ×

car-calc-alerts-matiushko 23:05 (38 хвилин тому)
You are receiving this email because your Amazon CloudWatch Alarm "car-calc-alb-latency-matiushko" in the US East (N. Virginia) region has entered the OK state,

car-calc-alerts-matiushko <no-reply@sns.amazonaws.com> 23:43 (0 хвилин тому)
кому мені

Схоже, мова цього повідомлення – англійська
Перекласти такою мовою: українська

You are receiving this email because your Amazon CloudWatch Alarm "car-calc-alb-latency-matiushko" in the US East (N. Virginia) region has entered the OK state, because "Threshold Crossed: 1 out of the last 1 datapoints [0.02346403718088245 (05/11/26 21:36:00)] was not greater than the threshold (2.0) (minimum 1 datapoint for ALARM -> OK transition)." at "Thursday 05 November, 2026 21:43:43 UTC".

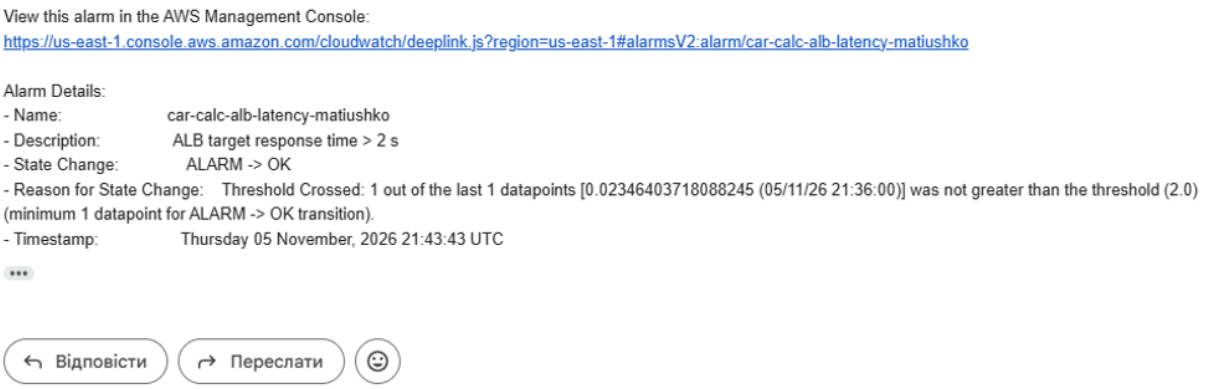


Рис. 34. Лист від SNS при переході алярму затримки в ALARM (тест 2): причина, фактичне значення метрики та посилання на алярм.

Рис. 35. Лист про відновлення (ALARM → OK) після повернення робочого порога 2 s — повний цикл алерту.

9.4. Витрати

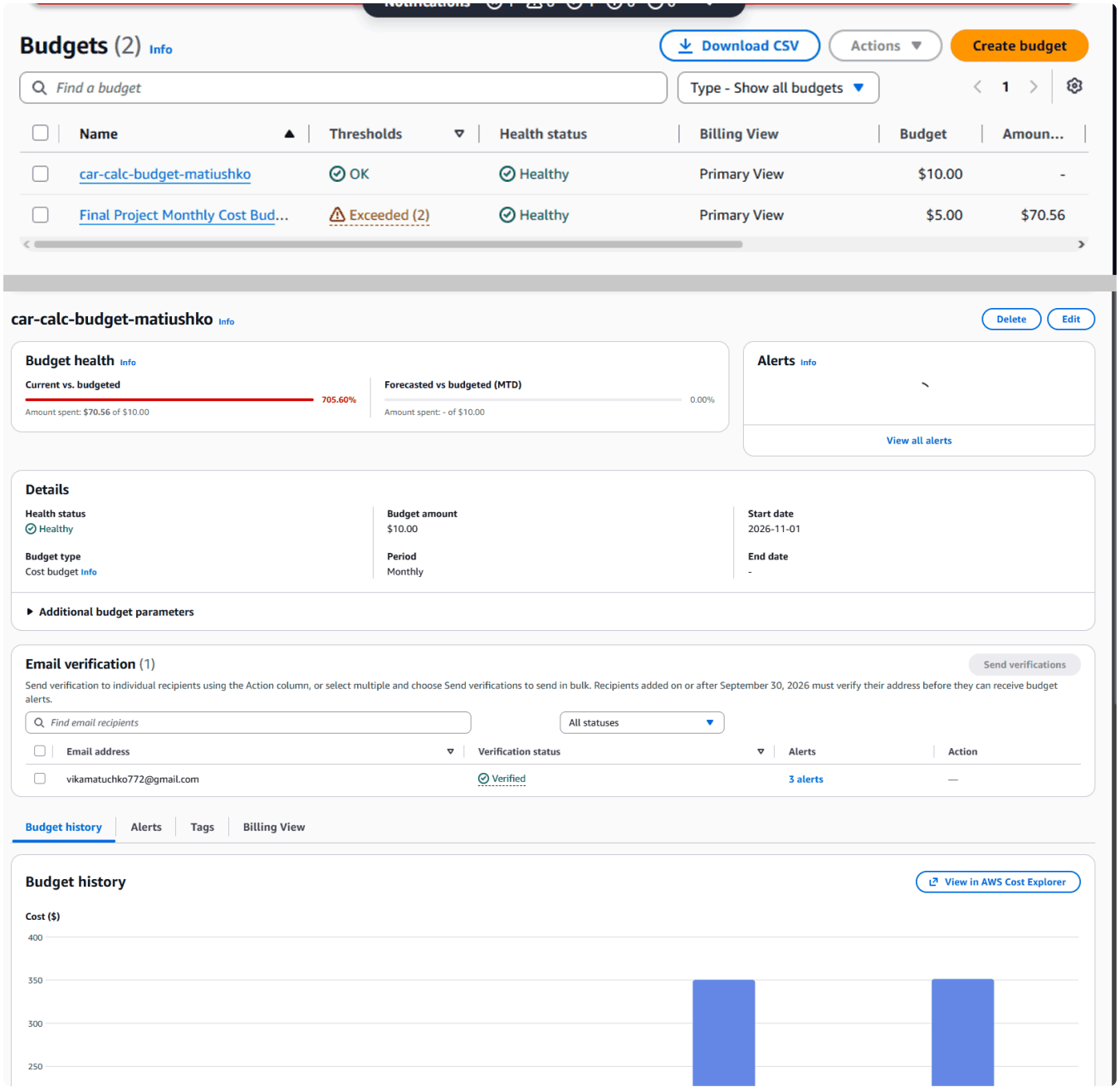


Рис. 36. AWS Budgets: створено власний бюджет `car-calc-budget-matiushko` ($10/міс) поруч ізі спільним бюджетом курсу.

Рис. 37. Деталі бюджету: 3 алерти, email-отримувач Verified. $70.56 — витрати всього спільного акаунта за листопад, а не лише цього проєкту.

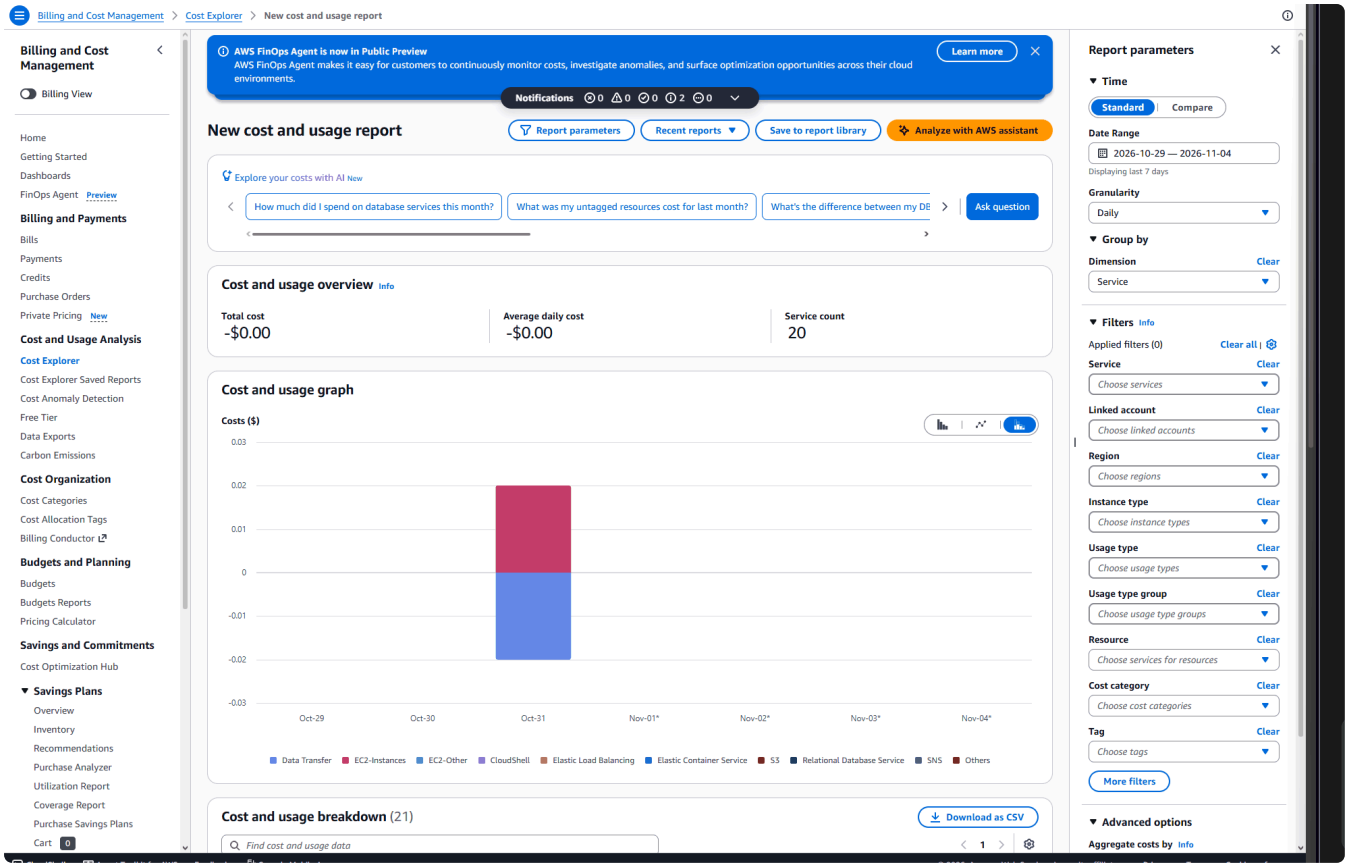


Рис. 38. Cost Explorer (7 днів, Group by Service): витрати акаунта за сервісами; підсумок ≈ $0, бо кредити курсу покривають споживання, а дані надходять із затримкою до доби.